UNIVERSITAS GADJAH MADA

SEKOLAH VOKASI

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK

LAPORAN UJIAN TENGAH SEMESTER

Mata Kuliah Pengembangan Perangkat Lunak

BUKU SERVIS

Aplikasi Pencatatan Servis Motor Berbasis Laravel, Vue, Docker, dan FrankenPHP

Disusun oleh : Rizendy Affarin

NIM : 24/535196/SV/24171

Kelas : PL5B1

Dosen : Galih Malela Damaraji

YOGYAKARTA

2026

ABSTRAK

Buku Servis adalah aplikasi web untuk mencatat riwayat servis motor. Setiap pengguna memiliki catatan sendiri yang berisi tanggal servis, odometer, jenis servis, biaya, bengkel, dan catatan. Laporan ini mendokumentasikan pembangunan aplikasi tersebut sebagai pemenuhan UTS mata kuliah Pengembangan Perangkat Lunak: REST API Laravel 12 dengan Sanctum, antarmuka Vue 3 yang memakai Axios, MySQL, Docker Compose, FrankenPHP sebagai server backend, serta pipeline GitHub Actions yang menguji, membangun, dan mendorong image ke GHCR hanya dari branch `main`.

Hasil pengujian menunjukkan 16 tes fitur lulus dengan 38 asersi. Seluruh endpoint selain `POST /api/login` menolak permintaan tanpa token dengan status 401. Pembuatan catatan mengembalikan 201, validasi mengembalikan 422, dan operasi baca, ubah, hapus mengembalikan 200. Ketiga container berjalan sehat; data tetap ada setelah `docker compose down` dan `docker compose up -d`. Backend terbukti dilayani FrankenPHP 1.13.3 dengan PHP 8.4.27, bukan `php artisan serve` atau PHP-FPM. Pipeline CI berjalan pada branch fitur, `dev`, dan pull request; job deploy hanya berjalan setelah push ke `main` dan menghasilkan image bertag commit SHA di GHCR.

Kata kunci: Laravel, Sanctum, Vue, Axios, Docker Compose, FrankenPHP, GitHub Actions, GHCR.

KATA PENGANTAR

Laporan ini disusun untuk mendokumentasikan pekerjaan UTS Pengembangan Perangkat Lunak yang berfokus pada integrasi backend, frontend, basis data, container, dan CI/CD. Aplikasi yang dibangun bernama Buku Servis. Masalah yang diangkat sederhana: catatan servis motor sering tersebar di buku manual, pesan chat, atau struk yang mudah hilang, sehingga pemilik motor sulit mengetahui kapan servis terakhir dilakukan dan berapa biaya yang sudah dikeluarkan.

Saya mengucapkan terima kasih kepada Bapak Galih Malela Damaraji selaku dosen pengampu, rekan-rekan kelas PL5B1, dan pihak-pihak yang membantu proses pengujian. Laporan ini berisi kode, tangkapan layar, keluaran perintah, serta kendala yang benar-benar ditemui selama pengerjaan. Bagian yang belum sempurna saya tulis apa adanya agar pembaca dapat menilai pekerjaan ini secara jujur.

Yogyakarta, 24 Oktober 2026

Rizendy Affarin

Daftar Isi

ABSTRAK **i**

KATA PENGANTAR **ii**

1 PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan 2
1.4 Ruang Lingkup 2
1.5 Manfaat 3
1.6 Sistematika Penulisan 3

2 LANDASAN TEORI **4**
2.1 Laravel dan Sanctum 4
2.2 Vue 3 dan Axios 4
2.3 Docker dan Docker Compose 4

2.4 FrankenPHP 5
2.5 GitHub Actions, GHCR, dan Branch Protection 5
2.6 Arsitektur Sistem 5
2.7 Alur Data 6

3 ANALISIS DAN PERANCANGAN 7
3.1 Kebutuhan Fungsional 7
3.2 Kebutuhan Nonfungsional 8
3.3 Model Data 8
3.4 Rancangan API 9
3.5 Rancangan Antarmuka 9
3.6 Rancangan Deployment 10
3.7 Rancangan CI/CD dan Strategi Branch 10
3.8 Skenario Pengujian 11

4 IMPLEMENTASI 12
4.1 Lingkungan Pengembangan 12
4.2 Struktur Repositori 12
4.3 Backend Laravel 13
4.3.1 Routing dan Perlindungan Token 13
4.3.2 Autentikasi 13
4.3.3 CRUD dan Kepemilikan Data 14
4.3.4 Validasi 15
4.3.5 Migrasi dan Seeder 16
4.3.6 Penanganan Permintaan Tanpa Token 16
4.4 Frontend Vue 17
4.4.1 Konfigurasi Alamat API 17
4.4.2 State Autentikasi dan Router 17
4.4.3 Formulir dan Daftar Catatan 18
4.5 Docker 18
4.5.1 Backend: FrankenPHP Multi-stage 18
4.5.2 Frontend: Node Build dan Nginx Runtime 20
4.5.3 Docker Compose 21
4.5.4 `.dockerignore` dan Rahasia 22
4.6 CI/CD 22
4.7 Cara Menjalankan 22

5 HASIL DAN PEMBAHASAN 24
5.1 Metode Pengujian 24
5.2 CPMK 1: REST API, Autentikasi, dan CRUD 24
5.2.1 Hasil Tes Otomatis 24
5.2.2 Verifikasi API Manual 25
5.2.3 Postman Collection 25
5.3 CPMK 2: Frontend Vue dan Integrasi Axios 26
5.4 CPMK 3: Docker Compose, FrankenPHP, dan Persistensi 35
5.4.1 Status Container 35
5.4.2 Bukti FrankenPHP 35
5.4.3 Persistensi Volume 36
5.5 CPMK 4: Git, Branch Protection, dan CI/CD 36
5.5.1 Repositori dan Branch 36
5.5.2 Branch Protection 40

5.5.3 GitHub Secrets . . . 42
5.5.4 Hasil Pipeline . . . 43
5.5.5 Image di GHCR . . . 45
5.6 CPMK 5: Dokumentasi dan Reproduksibilitas . . . 47
5.7 Pembahasan Alur Data End-to-End . . . 48
5.8 Kendala dan Solusi . . . 49
5.9 Kesesuaian dengan Ketentuan . . . 50

6 KESIMPULAN DAN SARAN **51**
6.1 Kesimpulan . . . 51
6.2 Saran . . . 51

DAFTAR PUSTAKA **53**

LAMPIRAN **55**
A.1 Keluaran Tes . . . 55
A.2 Keluaran `docker compose ps` . . . 56
A.3 Bukti Log FrankenPHP . . . 56
A.4 Bukti Persistensi . . . 57
A.5 Transkrip Verifikasi API . . . 57
A.6 Tautan Penting . . . 58

Daftar Gambar

Gambar 2.1 Arsitektur runtime aplikasi Buku Servis . . . 6
Gambar 2.2 Alur penambahan catatan servis dari Vue sampai MySQL . . . 6

Gambar 5.1 Respons login di Postman dan penyimpanan token . . . 26
Gambar 5.2 Respons pembuatan catatan dengan status 201 . . . 26
Gambar 5.3 Respons validasi dengan status 422 . . . 26
Gambar 5.4 Respons tanpa token dengan status 401 . . . 26
Gambar 5.5 Halaman masuk sebelum autentikasi . . . 27
Gambar 5.6 Halaman daftar catatan setelah login . . . 28
Gambar 5.7 Formulir tambah catatan yang sudah diisi . . . 29
Gambar 5.8 Catatan baru muncul di daftar setelah disimpan . . . 30
Gambar 5.9 Formulir ubah dengan data catatan terpilih . . . 31
Gambar 5.10 Daftar setelah biaya diubah . . . 32
Gambar 5.11 Konfirmasi hapus dua langkah . . . 33
Gambar 5.12 Daftar setelah catatan dihapus . . . 34
Gambar 5.13 Halaman utama repositori publik . . . 37
Gambar 5.14 Daftar branch `main`, `dev`, dan fitur . . . 38
Gambar 5.15 Dua pull request yang sudah digabung . . . 39
Gambar 5.16 PR #1: fitur Docker dan FrankenPHP masuk ke `dev` . . . 39
Gambar 5.17 PR #2: `dev` masuk ke `main` . . . 40
Gambar 5.18 Branch protection pada `main` . . . 41
Gambar 5.19 Branch protection pada `dev` . . . 42

Gambar 5.20 Daftar GitHub Secrets tanpa nilai rahasia 43
Gambar 5.21 Run pada `dev`: dua job lulus, deploy tidak muncul 44
Gambar 5.22 Run pada `main`: backend-test, frontend-build, dan deploy lulus 45
Gambar 5.23 Daftar package container di GHCR 46
Gambar 5.24 Halaman package backend dan tag SHA 47

Daftar Tabel

Tabel 3.1 Kebutuhan fungsional aplikasi 7
Tabel 3.2 Kebutuhan nonfungsional dan cara pemenuhannya 8
Tabel 3.3 Struktur tabel `service_logs` 8
Tabel 3.4 Rancangan endpoint API 9
Tabel 3.5 Service pada Docker Compose 10
Tabel 3.6 Perilaku pipeline berdasarkan event 10

Tabel 4.1 Versi perkakas pengembangan dan runtime 12

Tabel 5.1 Hasil verifikasi status HTTP API 25
Tabel 5.2 Status container dari `docker compose ps` 35
Tabel 5.3 Run GitHub Actions yang diamati 43
Tabel 5.4 Kendala yang ditemui dan solusinya 49
Tabel 5.5 Pemetaan ketentuan tugas ke bukti 50

DAFTAR KODE

4.1 Struktur direktori utama repositori . . . 12
4.2 Routing API di `backend/routes/api.php` . . . 13
4.3 Login dan logout pada `AuthController` . . . 14
4.4 Inti CRUD pada `ServiceLogController` . . . 14
4.5 Aturan validasi catatan servis . . . 15
4.6 Potongan migrasi `service_logs` . . . 16
4.7 Perbaikan redirect tamu pada `bootstrap/app.php` . . . 16
4.8 Instance Axios dan interceptor token . . . 17
4.9 Guard router dan penyimpanan sesi . . . 17
4.10 Bagian penting Dockerfile backend . . . 18
4.11 Konfigurasi FrankenPHP pada `docker/Caddyfile` . . . 19
4.12 Entrypoint backend . . . 19
4.13 Dockerfile frontend dua tahap . . . 20
4.14 Proxy `/api/` pada Nginx . . . 20
4.15 Ringkasan service pada `docker-compose.yml` . . . 21
4.16 Kondisi job deploy . . . 22
4.17 Penentuan nama image GHCR . . . 22
4.18 Menjalankan seluruh sistem dari nol . . . 22
5.1 Keluaran `php artisan test` (ringkas) . . . 24
5.2 Contoh respons validasi 422 . . . 25
5.3 Potongan bukti log FrankenPHP . . . 35
5.4 Hasil uji persistensi . . . 36
A.1 Keluaran lengkap `php artisan test` . . . 55
A.2 Keluaran `docker compose ps` setelah semua container sehat . . . 56
A.3 Startup dan access log FrankenPHP (tanpa warna) . . . 56
A.4 Transkrip uji persistensi volume . . . 57
A.5 Transkrip curl; token disamarkan . . . 57

Bab 1

PENDAHULUAN

1.1 Latar Belakang

Motor adalah kendaraan yang perawatannya bergantung pada catatan. Penggantian oli mesin, kampas rem, ban, aki, dan servis CVT punya interval yang berbeda. Tanpa catatan, pemilik motor cenderung mengandalkan ingatan atau menebak dari suara mesin. Akibatnya servis bisa terlambat, biaya membengkak, dan riwayat perawatan tidak bisa dijelaskan saat motor dijual.

Aplikasi pencatatan sederhana dapat menyelesaikan masalah tersebut asalkan dapat diakses dari browser, menyimpan data secara permanen, dan membatasi data berdasarkan pengguna. Karena itu proyek ini tidak hanya berhenti pada CRUD di satu komputer. Aplikasi harus berjalan sebagai beberapa container yang saling terhubung, memakai basis data MySQL, memiliki antarmuka Vue, dan melewati pipeline CI/CD. Persyaratan tersebut memaksa saya berlatih pada hal yang sering terlewat dalam tugas CRUD biasa: konfigurasi jaringan antar-container, migrasi basis data saat container mulai, penyimpanan volume, penanganan token, serta pemisahan rahasia dari repositori.

Aplikasi yang dibangun diberi nama **Buku Servis**. Entitas utamanya adalah catatan servis dengan field motor, tanggal servis, odometer, jenis servis, biaya, bengkel, dan catatan. Setiap catatan dimiliki oleh satu pengguna. Autentikasi memakai Laravel Sanctum dengan token bearer. Registrasi publik tidak dibuka karena ketentuan tugas menyatakan semua endpoint selain login harus memakai token; akun demo dibuat oleh seeder saat backend pertama kali berjalan.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, masalah yang diselesaikan dalam laporan ini adalah:

1. Bagaimana membangun REST API Laravel yang seluruh endpoint selain login dilindungi token Sanctum dan mengembalikan status HTTP yang sesuai?
2. Bagaimana menghubungkan frontend Vue 3 ke API tanpa menulis alamat API di dalam komponen?
3. Bagaimana menjalankan backend, frontend, dan MySQL melalui Docker Compose dengan jaringan service-name dan volume permanen?
4. Bagaimana memastikan backend benar-benar dilayani FrankenPHP, bukan `php artisan serve` atau PHP-FPM?

5. Bagaimana menyusun alur Git dan GitHub Actions sehingga pengujian berjalan di setiap branch, sedangkan deploy hanya terjadi dari `main`?

1.3 Tujuan

Tujuan yang ingin dicapai:

1. Menyediakan API CRUD catatan servis yang aman dan konsisten.
2. Menyediakan antarmuka login dan pengelolaan catatan servis yang memakai token dari API.
3. Menjalankan seluruh sistem dengan satu perintah Docker Compose dan memastikan data bertahan setelah container dihentikan.
4. Membuktikan penggunaan FrankenPHP melalui konfigurasi image dan log runtime.
5. Menerapkan CI/CD, branch protection, serta penyimpanan rahasia di GitHub Secrets.
6. Mendokumentasikan proses, bukti pengujian, dan kendala secara jujur.

1.4 Ruang Lingkup

Ruang lingkup aplikasi dibatasi pada:

- Satu resource CRUD, yaitu `service_logs` (catatan servis motor).
- Autentikasi token melalui `POST /api/login`, `GET /api/me`, dan `POST /api/logout`. Tidak ada registrasi publik.
- Frontend satu halaman untuk login dan satu halaman untuk daftar, tambah, ubah, dan hapus catatan.
- Basis data MySQL 8.4 dengan volume bernama.
- Backend Laravel 12 + Sanctum yang dilayani FrankenPHP.
- CI/CD GitHub Actions dengan job pengujian, build, dan deploy ke GHCR.

Di luar ruang lingkup: aplikasi mobile native, notifikasi pengingat servis, multi-bahasa, pembayaran, dan manajemen bengkel multi-cabang.

1.5 Manfaat

Secara praktis, aplikasi ini memberi cara cepat untuk mencatat servis dan melihat total biaya perawatan. Secara akademis, proyek ini menjadi latihan integrasi antara API, SPA, basis data, container, dan pipeline. Dari sisi dokumentasi, laporan ini menyimpan bukti eksekusi yang dapat diperiksa ulang: keluaran tes, status HTTP, log FrankenPHP, keluaran `docker compose ps`, dan tautan commit.

1.6 Sistematika Penulisan

BAB I Pendahuluan
memuat latar belakang, rumusan masalah, tujuan, ruang lingkup, manfaat, dan sistematika.

BAB II Landasan Teori
membahas Laravel, Sanctum, Vue, Axios, Docker, FrankenPHP, dan GitHub Actions.

BAB III Analisis dan Perancangan
menjelaskan kebutuhan, model data, rancangan API, rancangan antarmuka, dan rancangan deployment.

BAB IV Implementasi
memaparkan kode dan konfigurasi yang ditulis.

BAB V Hasil dan Pembahasan
menyajikan bukti pengujian per CPMK, alur data, serta kendala dan solusinya.

BAB VI Kesimpulan dan Saran
merangkum hasil dan memberi saran lanjutan.

Bab 2

LANDASAN TEORI

2.1 Laravel dan Sanctum

Laravel adalah framework PHP dengan pola MVC. Pada versi 12, konfigurasi aplikasi dipusatkan di `bootstrap/app.php`; routing API dipisahkan dari routing web. Permintaan API masuk melalui `public/index.php`, melewati middleware global, middleware grup, lalu router. Controller mengembalikan respons JSON. Eloquent menjembatani model dengan tabel MySQL.

Sanctum menyediakan token akses pribadi. Saat login berhasil, server membuat token dan menyimpan hash-nya di tabel `personal_access_tokens`. Token asli hanya dikirim sekali ke klien. Pada permintaan berikutnya, klien mengirim header `Authorization: Bearer <token>`. Middleware `auth:sanctum` memeriksa token tersebut. Jika tidak ada atau tidak valid, permintaan ditolak dengan 401.

2.2 Vue 3 dan Axios

Vue 3 dengan Composition API memakai `ref`, `computed`, dan `onMounted` untuk mengelola state halaman. Vue Router mengatur navigasi `/masuk` dan `/` serta menjaga halaman daftar agar hanya dapat dibuka saat token tersedia. Axios dipakai sebagai klien HTTP karena mendukung instance dengan `baseURL` dan interceptor.

Interceptor request menambahkan token ke setiap permintaan. Interceptor response menangani 401 dengan memicu event `auth:expired`, sehingga aplikasi dapat kembali ke halaman masuk ketika token dicabut. Alamat API diambil dari `import.meta.env.VITE_API_URL`; nilai ini diisi saat build, bukan ditulis di dalam komponen.

2.3 Docker dan Docker Compose

Docker mengemas aplikasi beserta dependensinya ke dalam image. Container adalah instance image yang berjalan. Docker Compose mendefinisikan beberapa service dalam satu berkas YAML. Setiap service dapat memiliki image, build context, environment, port, volume, healthcheck, dan network.

Dua hal penting dalam proyek ini adalah *multi-stage build* dan *named volume*. Multi-stage build memisahkan tahap kompilasi dari tahap runtime sehingga image akhir tidak membawa compiler atau `node_modules`. Named volume menyimpan data MySQL di luar siklus hidup

container, sehingga data tidak hilang saat container dihapus.

2.4 FrankenPHP

FrankenPHP adalah server aplikasi PHP yang dibangun di atas Caddy. Ia dapat menjalankan PHP sebagai worker dan menangani HTTP secara langsung. Pada proyek ini, FrankenPHP memakai konfigurasi Caddy untuk melayani direktori `public/` Laravel pada port 8000. Pilihan ini berbeda dari `php artisan serve` yang hanya untuk pengembangan, dan berbeda dari PHP-FPM yang memerlukan web server terpisah.

FrankenPHP juga menulis access log ke stdout. Log tersebut dapat dibaca dengan `docker compose logs backend` dan menjadi bukti bahwa permintaan benar-benar melewati server tersebut.

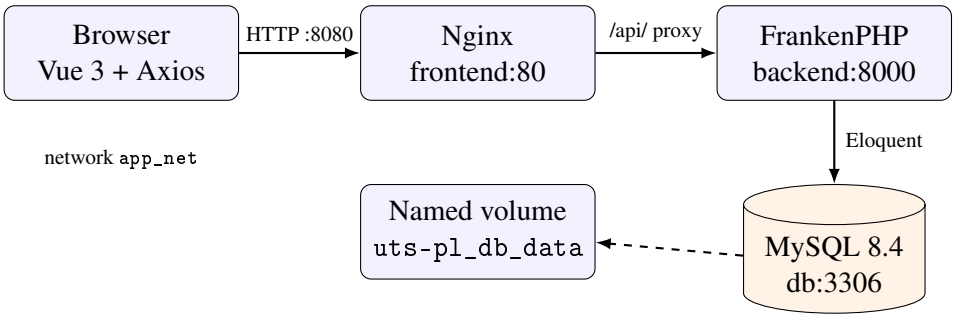
2.5 GitHub Actions, GHCR, dan Branch Protection

GitHub Actions menjalankan workflow berdasarkan event. Pada proyek ini event yang dipakai adalah `push` ke `main`, `dev`, dan `feature/**`, serta `pull_request` ke `main` dan `dev`. Job `backend-test` menjalankan `php artisan test`; job `frontend-build` menjalankan `npm ci` dan `npm run build`; job `deploy` hanya berjalan pada push ke `main`.

GHCR (GitHub Container Registry) menyimpan image container. Image diberi tag commit SHA, bukan `latest`, agar versi yang di-deploy dapat dilacak kembali ke commit. Branch protection mencegah push langsung ke `main` dan `dev`; perubahan harus melalui pull request dan status check harus lulus.

2.6 Arsitektur Sistem

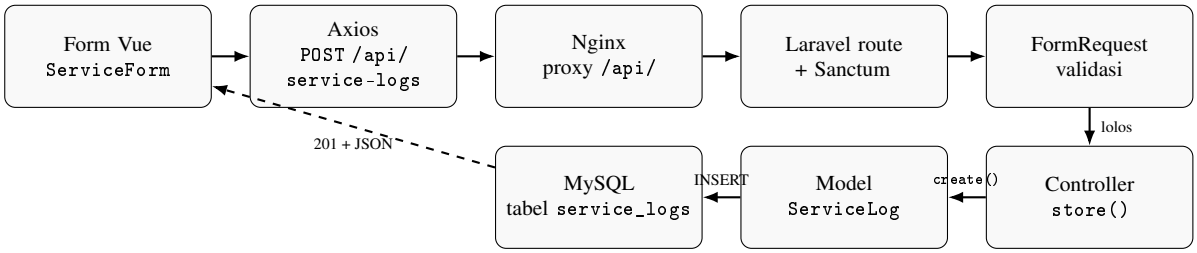
Gambar 2.1 menunjukkan hubungan antar-komponen. Browser memuat frontend dari Nginx pada port 8080. Permintaan ke `/api/` diteruskan Nginx ke service `backend:8000`. Backend memakai FrankenPHP, memproses route Laravel, dan mengakses MySQL melalui service `db:3306`. Semua service berada pada network `app_net`; nama service dipakai sebagai alamat, bukan IP container.



Gambar 2.1: Arsitektur runtime aplikasi Buku Servis

2.7 Alur Data

Alur data dari klik pengguna sampai data tersimpan dijelaskan pada Gambar 2.2. Contoh yang dipakai adalah penambahan catatan servis.



Gambar 2.2: Alur penambahan catatan servis dari Vue sampai MySQL

Jika validasi gagal, `FormRequest` menghentikan proses dan Laravel mengembalikan 422 berisi `errors` per field. Jika token tidak ada, middleware Sanctum mengembalikan 401 sebelum controller dijalankan. Jika token valid tetapi catatan milik pengguna lain, query `findOrFail` menghasilkan 404. Pola ini membuat respons dapat dibedakan oleh frontend dan oleh penguji API.

Bab 3

ANALISIS DAN PERANCANGAN

3.1 Kebutuhan Fungsional

Tabel 3.1: Kebutuhan fungsional aplikasi

No	Kebutuhan	Bukti
F-01	Pengguna dapat masuk dengan email dan kata sandi, lalu menerima token.	`POST /login`
F-02	Pengguna dapat keluar dan token aktif dicabut.	`POST /logout`
F-03	Pengguna dapat melihat daftar catatan servis miliknya.	`GET /service-logs`
F-04	Pengguna dapat menambah catatan servis.	`POST /service-logs`
F-05	Pengguna dapat melihat detail satu catatan.	`GET /service-logs/{id}`
F-06	Pengguna dapat mengubah catatan.	`PUT /service-logs/{id}`
F-07	Pengguna dapat menghapus catatan.	`DELETE /service-logs/{id}`
F-08	Data pengguna lain tidak dapat dibaca atau diubah.	query terbatas `user_id`
F-09	Frontend menampilkan ringkasan odometer dan total biaya.	halaman daftar
F-10	Aplikasi menolak input tidak valid dengan pesan per field.	422 + `errors`

Seluruh path pada kolom Bukti diawali `/api`.

3.2 Kebutuhan Nonfungsional

Tabel 3.2: Kebutuhan nonfungsional dan cara pemenuhannya

Aspek	Ketentuan
Keamanan	Token Sanctum; tidak ada registrasi publik; rahasia hanya di GitHub Secrets dan berkas `.env` yang tidak di-commit.
Keterlacakan	Image GHCR diberi tag commit SHA, bukan `latest`.
Portabilitas	Seluruh sistem berjalan dengan Docker Compose; tidak perlu PHP/Node di host.
Konsistensi data	Validasi server-side; tipe kolom MySQL; `user_id` wajib.
Ketersediaan	Healthcheck backend `/up` dan frontend `/`; restart `unless-stopped`.
Kemudahan operasi	Satu perintah `docker compose up -d -build`; migrasi otomatis dari entrypoint.
Kualitas kode	PSR-4, FormRequest, service layer tipis, test fitur.
Dokumentasi	README, Postman Collection, komentar pada konfigurasi non-obvious.

3.3 Model Data

Aplikasi memakai tiga tabel utama: `users`, `service_logs`, dan `personal_access_tokens`. Tabel `users` dimiliki Laravel sejak awal. Tabel `service_logs` menyimpan catatan servis dan memiliki foreign key `user_id`. Tabel `personal_access_tokens` menyimpan token Sanctum.

Tabel 3.3: Struktur tabel `service_logs`

Kolom	Tipe	Keterangan
`id`	bigint unsigned	Primary key, auto increment.
`user_id`	bigint unsigned	Foreign key ke `users.id`, cascade delete.
`motor`	varchar(100)	Nama atau tipe motor.
`serviced_at`	date	Tanggal servis, format `YYYY-MM-DD`.
`odometer_km`	int unsigned	Kilometer saat servis.
`service_type`	varchar(100)	Jenis pekerjaan servis.
`cost`	int unsigned	Biaya dalam rupiah.
`workshop`	varchar(100) nullable	Nama bengkel.
`notes`	text nullable	Catatan tambahan.
`created_at`, `updated_at`	timestamp	Audit waktu.

Relasi yang dipakai: satu `User` memiliki banyak `ServiceLog`; satu `ServiceLog` dimiliki satu `User`. Indeks gabungan `(user_id, serviced_at)` membantu query daftar yang selalu difilter berdasarkan pengguna dan diurutkan berdasarkan tanggal.

3.4 Rancangan API

API mengikuti gaya REST dengan prefix `/api`. Tabel 3.4 merangkum endpoint, autentikasi, dan status yang diharapkan.

Tabel 3.4: Rancangan endpoint API

Method	Endpoint	Keterangan	Status
POST	`/api/login`	Autentikasi, menghasilkan token	200/401/422
GET	`/api/me`	Profil pemilik token	200/401
POST	`/api/logout`	Mencabut token aktif	200/401
GET	`/api/service-logs`	Daftar catatan milik pengguna	200/401
POST	`/api/service-logs`	Menambah catatan	201/401/422
GET	`/api/service-logs/{id}`	Detail catatan milik pengguna	200/401/404
PUT	`/api/service-logs/{id}`	Mengubah catatan	200/401/404/422
DELETE	`/api/service-logs/{id}`	Menghapus catatan	200/401/404

Aturan validasi yang dipakai:

- `motor`: wajib, teks, maksimal 100 karakter.
- `serviced_at`: wajib, format tanggal `Y-m-d`.
- `odometer_km`: wajib, bilangan bulat, 0–9.999.999.
- `service_type`: wajib, teks, maksimal 100 karakter.
- `cost`: wajib, bilangan bulat, 0–999.999.999.
- `workshop`: opsional, teks, maksimal 100 karakter.
- `notes`: opsional, teks, maksimal 1.000 karakter.

Pesan validasi disimpan di `App\Support\ValidationMessages` agar controller dan FormRequest memakai pesan yang sama. Untuk aturan ukuran (`min`/`max`) pada kolom numerik, pesan diberikan per atribut karena Laravel mengambil pesan size dari translator, bukan dari larik `messages()` biasa. Detail ini sempat menjadi kendala dan dibahas pada Subbab 5.8.

3.5 Rancangan Antarmuka

Ada dua halaman utama:

1. **Halaman masuk** (`/masuk`) berisi email, kata sandi, pesan error 401/422, dan tombol masuk. Halaman ini tidak menampilkan tautan registrasi karena registrasi publik ditutup.

2. **Halaman catatan** (`/`) berisi ringkasan odometer, total biaya, formulir catat/ubah, dan daftar riwayat servis. Setiap baris memiliki tombol Ubah dan Hapus. Hapus memakai konfirmasi dua langkah.

Komponen `ServiceForm` dipakai untuk tambah dan ubah. Saat prop `editing` berisi catatan, formulir terisi data tersebut dan tombol berubah menjadi "Simpan perubahan". Saat batal, formulir kembali kosong. Ringkasan odometer memakai komponen `Odometer` yang menampilkan angka besar dan total biaya dihitung dari daftar yang sedang tampil.

3.6 Rancangan Deployment

Tabel 3.5: Service pada Docker Compose

Service	Image	Peran
`db`	`mysql:8.4.13`	Basis data, volume `uts-pl_db_data`, healthcheck `mysqladmin ping`.
`backend`	`uts-pl-backend:local`	Laravel + FrankenPHP pada port 8000, healthcheck `/up`.
`frontend`	`uts-pl-frontend:local`	Nginx pada port 80, diteruskan ke host 8080, proxy `/api/`.

Kredensial MySQL diambil dari `.env` yang tidak di-commit. Berkas `.env.example` menyediakan nama variabel tanpa nilai rahasia. Backend menunggu `db` sehat sebelum mulai, lalu entrypoint menjalankan migrasi dengan percobaan ulang. Frontend dibangun dengan argumen `VITE_API_URL=/api`, sehingga browser memanggil origin yang sama dan tidak ada masalah CORS.

3.7 Rancangan CI/CD dan Strategi Branch

Tabel 3.6: Perilaku pipeline berdasarkan event

Event	Job yang berjalan	Deploy
Push ke `feature/**`	`backend-test`, `frontend-build`	Tidak
Pull request ke `dev`	`backend-test`, `frontend-build`	Tidak
Push ke `dev`	`backend-test`, `frontend-build`	Tidak
Pull request ke `main`	`backend-test`, `frontend-build`	Tidak
Push ke `main`	`backend-test`, `frontend-build`, `deploy`	Ya, GHCR

Branch yang dipakai: `main` sebagai rilis, `dev` sebagai integrasi, dan `feature/docker-frankenphp` sebagai tempat pekerjaan fitur. Perubahan masuk mela-

lui pull request. Branch protection pada `main` dan `dev` mewajibkan pull request, status check `backend-test` dan `frontend-build`, branch up-to-date, serta berlaku untuk administrator.

3.8 Skenario Pengujian

Pengujian dilakukan pada lima lapisan:

1. **Unit/feature test** dengan `php artisan test` untuk autentikasi, otorisasi, validasi, dan CRUD.
2. **API manual** dengan curl dan Postman untuk memeriksa status HTTP serta bentuk JSON.
3. **Antarmuka** dengan browser: login, tambah, ubah, hapus, dan kembali ke halaman masuk setelah logout.
4. **Container** dengan `docker compose ps`, log backend, dan uji persistensi volume.
5. **Pipeline** dengan push ke branch, pull request, dan push ke `main`; status job dan image GHCR diperiksa.

Bab 4

IMPLEMENTASI

4.1 Lingkungan Pengembangan

Pekerjaan dilakukan pada Windows 11 dengan Docker Desktop. Versi perkakas yang tercatat saat pengerjaan ditunjukkan pada Tabel 4.1. Versi di host berbeda dari versi di container; yang menjadi acuan runtime adalah versi di dalam image.

Tabel 4.1: Versi perkakas pengembangan dan runtime

Perkakas	Versi	Keterangan
Docker Engine	28.5.1	Docker Desktop di Windows.
Docker Compose	v2.40.3-desktop.1	Perintah `docker compose`.
Git	2.45.1	Windows.
PHP host	8.2.12	Untuk menjalankan tes lokal.
Composer host	2.8.11	Untuk dependensi lokal.
Node host	22.19.0	Untuk build frontend lokal.
PHP container	8.4.27	Dari image FrankenPHP.
Composer container	2.10.3	Dari image `composer:2.10.3`.
Node container	22.23.4	Tahap build frontend.
Nginx container	1.30.6	Tahap runtime frontend.
MySQL	8.4.13	Service `db`.

4.2 Struktur Repositori

Repositori dipisahkan menjadi `backend`, `frontend`, `postman`, `laporan`, dan berkas orkestrasi di akar. Pemisahan ini membuat build context Docker tetap kecil dan mencegah frontend ikut masuk ke image backend.

```
.
|-- .github/workflows/ci.yml
|-- backend/                 # Laravel 12, Sanctum, FrankenPHP
|   |-- app/Http/Controllers
|   |-- app/Http/Requests
|   |-- app/Models
|   |-- docker/Caddyfile
|   |-- docker/entrypoint.sh
|   |-- Dockerfile
|   |-- routes/api.php
```

```
|   `-- tests/Feature
|-- frontend/                # Vue 3, Vite, Axios, Nginx
|   |-- docker/nginx.conf
|   |-- src/api.js
|   |-- src/views
|   |-- Dockerfile
|   `-- package.json
|-- postman/buku-servis.postman_collection.json
|-- laporan/
|-- docker-compose.yml
|-- .env.example
`-- README.md
```

Kode 4.1: Struktur direktori utama repositori

4.3 Backend Laravel

4.3.1 Routing dan Perlindungan Token

Berkas `routes/api.php` sengaja pendek. Hanya login yang berada di luar grup `auth:sanctum`. Semua route lain, termasuk `/me` dan `/logout`, berada di dalam grup. Tidak ada route registrasi.

```php
<?php

use App\Http\Controllers\AuthController;
use App\Http\Controllers\ServiceLogController;
use Illuminate\Support\Facades\Route;

// Hanya login yang dapat diakses tanpa token.
Route::post('/login', [AuthController::class, 'login']);

Route::middleware('auth:sanctum')->group(function () {
    Route::get('/me', [AuthController::class, 'me']);
    Route::post('/logout', [AuthController::class, 'logout']);

    Route::apiResource('service-logs', ServiceLogController::class);
});
```

Kode 4.2: Routing API di `backend/routes/api.php`

4.3.2 Autentikasi

`AuthController::login()` memvalidasi email dan kata sandi, mencari pengguna, dan memeriksa hash kata sandi. Jika gagal, respons 401 dengan pesan yang tidak membocorkan apakah email atau kata sandi yang salah. Jika berhasil, token dibuat dengan nama `spa` dan dikirim bersama data pengguna.

```php
public function login(Request $request): JsonResponse
{
    $data = $request->validate([
        'email' => ['required', 'email'],
        'password' => ['required', 'string'],
    ], ValidationMessages::messages(), ValidationMessages::attributes());

    $user = User::where('email', $data['email'])->first();

    if (! $user || ! Hash::check($data['password'], $user->password)) {
        return response()->json(['message' => 'Email atau kata sandi salah.'], 401);
    }

    return response()->json([
        'user' => $user,
        'token' => $user->createToken('spa')->plainTextToken,
    ]);
}

public function logout(Request $request): JsonResponse
{
    $request->user()->currentAccessToken()->delete();

    return response()->json(['message' => 'Berhasil keluar.']);
}
```

Kode 4.3: Login dan logout pada `AuthController`

4.3.3 CRUD dan Kepemilikan Data

`ServiceLogController` memakai relasi `$request->user()->serviceLogs()` untuk semua query. Cara ini lebih aman daripada menerima `user_id` dari request karena pengguna tidak dapat memilih pemilik catatan. Method `findOwned()` memakai `findOrFail` pada relasi, sehingga catatan milik pengguna lain menghasilkan 404, bukan 403 yang membocorkan keberadaan data.

```php
public function index(Request $request): JsonResponse
{
    $logs = $request->user()
        ->serviceLogs()
        ->orderByDesc('serviced_at')
        ->orderByDesc('id')
        ->get();

    return response()->json(['data' => $logs]);
}

public function store(ServiceLogRequest $request): JsonResponse
```

```
{
    $log = $request->user()->serviceLogs()->create($request->validated());

    return response()->json(['data' => $log], 201);
}

public function update(ServiceLogRequest $request, string $id): JsonResponse
{
    $log = $this->findOwned($request, $id);
    $log->update($request->validated());

    return response()->json(['data' => $log]);
}

public function destroy(Request $request, string $id): JsonResponse
{
    $this->findOwned($request, $id)->delete();

    return response()->json(['message' => 'Catatan dihapus.']);
}

private function findOwned(Request $request, string $id)
{
    return $request->user()->serviceLogs()->findOrFail($id);
}
```

Kode 4.4: Inti CRUD pada `ServiceLogController`

4.3.4 Validasi

Validasi ditempatkan di `ServiceLogRequest` agar controller tetap ringkas dan aturan yang sama dipakai untuk `store` dan `update`. Pesan berbahasa Indonesia dipusatkan di `ValidationMessages`.

```
public function rules(): array
{
    return [
        'motor' => ['required', 'string', 'max:100'],
        'serviced_at' => ['required', 'date_format:Y-m-d'],
        'odometer_km' => ['required', 'integer', 'min:0', 'max:9999999'],
        'service_type' => ['required', 'string', 'max:100'],
        'cost' => ['required', 'integer', 'min:0', 'max:999999999'],
        'workshop' => ['nullable', 'string', 'max:100'],
        'notes' => ['nullable', 'string', 'max:1000'],
    ];
}
```

Kode 4.5: Aturan validasi catatan servis

4.3.5 Migrasi dan Seeder

Migrasi membuat tabel `service_logs` dengan foreign key dan indeks gabungan. `DemoSeeder` membuat akun `demo@example.com` / `password123` dan lima catatan contoh. Seeder dipanggil dari entrypoint, bukan dari route registrasi. Dengan begitu, aplikasi tetap memenuhi aturan "semua endpoint selain login memerlukan token" tanpa mengorbankan kemudahan pengujian.

```
Schema::create('service_logs', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->string('motor', 100);
    $table->date('serviced_at');
    $table->unsignedInteger('odometer_km');
    $table->string('service_type', 100);
    $table->unsignedInteger('cost');
    $table->string('workshop', 100)->nullable();
    $table->text('notes')->nullable();
    $table->timestamps();

    $table->index(['user_id', 'serviced_at']);
});
```

Kode 4.6: Potongan migrasi `service_logs`

4.3.6 Penanganan Permintaan Tanpa Token

Saat pengujian container, ditemukan bahwa permintaan tanpa header `Accept: application/json` dari Nginx sempat menghasilkan 500, bukan 401. Penyebabnya adalah middleware `Authenticate` mencoba mengarahkan tamu ke route bernama `login` yang tidak ada. Perbaikannya adalah memberi tahu middleware bahwa aplikasi ini API-only dan tidak memiliki halaman login web.

```
->withMiddleware(function (Middleware $middleware): void {
    // Aplikasi ini API-only. Tamu tidak boleh diarahkan ke route login web
    // yang tidak ada; biarkan exception handler mengembalikan 401 JSON.
    $middleware->redirectGuestsTo(fn () => null);
})
```

Kode 4.7: Perbaikan redirect tamu pada `bootstrap/app.php`

Perbaikan ini disertai tes regresi `test_guest_without_json_accept_header_still_gets_401`. Kasus ini menunjukkan pentingnya menguji permintaan persis seperti yang dikirim proxy, bukan hanya `getJson()` yang selalu menambahkan header JSON.

4.4 Frontend Vue

4.4.1 Konfigurasi Alamat API

Alamat API tidak ditulis di komponen. Nilainya berasal dari `VITE_API_URL`. Di lokal, `frontend/.env` dapat berisi `http://localhost:8000/api`; di Docker, build arg diisi `/api` dan Nginx meneruskan permintaan ke `backend:8000`.

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  headers: { Accept: 'application/json' },
})

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

api.interceptors.response.use(
  (response) => response,
  (error) => {
    const url = error.config?.url ?? ''
    const isAuthCall = url === '/login' || url === '/register'
    if (error.response?.status === 401 && !isAuthCall) {
      window.dispatchEvent(new Event('auth:expired'))
    }
    return Promise.reject(error)
  },
)
```

Kode 4.8: Instance Axios dan interceptor token

4.4.2 State Autentikasi dan Router

Token dan data pengguna disimpan di `localStorage` agar sesi bertahan setelah halaman dimuat ulang. `auth.login()` memanggil API, menyimpan token, lalu router mengarahkan ke halaman catatan. `auth.logout()` tetap menghapus token lokal walaupun permintaan logout ke server gagal.

```
router.beforeEach((to) => {
  if (to.meta.auth && !auth.isLoggedIn) return '/masuk'
  if (to.meta.guest && auth.isLoggedIn) return '/'
})

function save({ token, user }) {
  state.token = token
```

```
  state.user = user
  localStorage.setItem('token', token)
  localStorage.setItem('user', JSON.stringify(user))
}
```

Kode 4.9: Guard router dan penyimpanan sesi

4.4.3 Formulir dan Daftar Catatan

Formulir memakai `watch` pada prop `editing`. Saat pengguna menekan Ubah, data catatan disalin ke form. Saat simpan, payload dikirim dengan method `PUT` jika sedang mengubah, atau `POST` jika menambah. Respons 422 dipetakan ke `errors` per field; respons 401 diserahkan ke interceptor.

Daftar catatan diurutkan dari tanggal terbaru. Ringkasan menampilkan odometer tertinggi dan total biaya dari catatan yang tampil. Konfirmasi hapus tidak memakai `window.confirm`, tetapi state `confirmingId` agar tampilan konsisten dan dapat diuji secara visual.

4.5 Docker

4.5.1 Backend: FrankenPHP Multi-stage

Dockerfile backend memiliki dua stage. Stage `vendor` memasang dependensi Composer. Stage `runtime` memakai image FrankenPHP yang sama, membuat user non-root `app`, menyalin hasil build, lalu menjalankan entrypoint. Perintah `composer install` sengaja dijalankan sebelum kode disalin agar layer dependensi tetap ter-cache saat hanya kode yang berubah.

```
FROM dunglas/frankenphp:1.13.3-php8.4-alpine AS vendor
RUN install-php-extensions pdo_mysql intl bcmath opcache
COPY --from=composer:2.10.3 /usr/bin/composer /usr/local/bin/composer
WORKDIR /app
COPY composer.json composer.lock ./
RUN composer install --no-dev --no-scripts --no-autoloader \
    --no-interaction --prefer-dist --no-progress
COPY . .
RUN composer dump-autoload --no-dev --optimize --classmap-authoritative

FROM dunglas/frankenphp:1.13.3-php8.4-alpine AS runtime
RUN install-php-extensions pdo_mysql intl bcmath opcache \
    && addgroup -S app && adduser -S -G app -h /app app
WORKDIR /app
COPY --from=vendor --chown=app:app /app /app
COPY --chown=app:app docker/Caddyfile /etc/caddy/Caddyfile
COPY --chown=app:app docker/entrypoint.sh /usr/local/bin/entrypoint
USER app
```

```
EXPOSE 8000
HEALTHCHECK CMD wget -q -O /dev/null http://127.0.0.1:8000/up || exit 1
ENTRYPOINT ["entrypoint"]
CMD ["frankenphp", "run", "--config", "/etc/caddy/Caddyfile"]
```

Kode 4.10: Bagian penting Dockerfile backend

Konfigurasi Caddy dibuat minimal: root di `/app/public`, `php_server`, kompresi, dan access log ke stdout. Tidak ada port admin yang dibuka.

```
{
    frankenphp
    order php_server before file_server
    admin off
}

:8000 {
    root * /app/public
    encode zstd br gzip
    log {
        output stdout
        format console
    }
    php_server
}
```

Kode 4.11: Konfigurasi FrankenPHP pada `docker/Caddyfile`

Entrypoint menunggu MySQL siap, menjalankan migrasi dengan maksimal 30 percobaan, menjalankan `DemoSeeder`, lalu membuat cache konfigurasi dan route. Pola ini membuat container backend tidak langsung gagal ketika MySQL belum menerima koneksi.

```
#!/bin/sh
set -eu

attempt=1
until php artisan migrate --force --no-interaction; do
    if [ "$attempt" -ge 30 ]; then
        echo "Migrasi gagal setelah $attempt percobaan." >&2
        exit 1
    fi
    echo "Basis data belum siap (percobaan $attempt/30), menunggu 2 detik..." >&2
    attempt=$((attempt + 1))
    sleep 2
done

php artisan db:seed --class=DemoSeeder --force --no-interaction
php artisan config:cache
php artisan route:cache

```

```
exec "$@"
```

Kode 4.12: Entrypoint backend

4.5.2 Frontend: Node Build dan Nginx Runtime

Frontend memakai dua stage. Stage `build` memakai Node 22.23.4, menyalin `package.json` dan `package-lock.json` lebih dulu, menjalankan `npm ci`, lalu `npm run build`. Stage runtime hanya menyalin `dist` ke Nginx. Tidak ada `node_modules` di image akhir.

```
FROM node:22.23.4-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --no-audit --no-fund
COPY . .
ARG VITE_API_URL=/api
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

FROM nginx:1.30.6-alpine3.24 AS runtime
COPY docker/nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
HEALTHCHECK CMD wget -q -O /dev/null http://127.0.0.1/ || exit 1
```

Kode 4.13: Dockerfile frontend dua tahap

Nginx memakai `resolver 127.0.0.11` dan variabel `$backend` agar alamat service `backend` diperbarui melalui DNS internal Docker. Penulisan IP container di konfigurasi dihindari karena IP dapat berubah saat container dibuat ulang.

```
resolver 127.0.0.11 valid=10s ipv6=off;
set $backend http://backend:8000;

location /api/ {
    proxy_pass $backend$request_uri;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Kode 4.14: Proxy `/api/` pada Nginx

4.5.3 Docker Compose

Berkas `docker-compose.yml` mendefinisikan tiga service dan satu volume. Backend menunggu `db` berstatus sehat. Frontend menunggu backend, tetapi tetap dapat mulai sebelum backend siap karena Nginx akan mengembalikan 502 sampai backend hidup; healthcheck frontend hanya memeriksa halaman statis.

```yaml
name: uts-pl

services:
  db:
    image: mysql:8.4.13
    environment:
      MYSQL_DATABASE: ${DB_DATABASE}
      MYSQL_USER: ${DB_USERNAME}
      MYSQL_PASSWORD: ${DB_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -u root -p\"
    $$MYSQL_ROOT_PASSWORD\" --silent"]

  backend:
    build: ./backend
    depends_on:
      db:
        condition: service_healthy
    environment:
      DB_HOST: db
      DB_DATABASE: ${DB_DATABASE}
      DB_USERNAME: ${DB_USERNAME}
      DB_PASSWORD: ${DB_PASSWORD}
    ports:
      - "8000:8000"

  frontend:
    build:
      context: ./frontend
      args:
        VITE_API_URL: /api
    ports:
      - "8080:80"

volumes:
  db_data:
```

Kode 4.15: Ringkasan service pada `docker-compose.yml`

4.5.4 `.dockerignore` dan Rahasia

Setiap build context memiliki `.dockerignore`. Backend mengabaikan `vendor`, `.env`, `storage/logs`, dan cache. Frontend mengabaikan `node_modules`, `dist`, dan `.env`. Tidak ada `ARG` atau `ENV` berisi kata sandi di Dockerfile. Nilai rahasia masuk saat runtime melalui environment Compose yang dibaca dari `.env` yang tidak di-commit.

4.6 CI/CD

Workflow `.github/workflows/ci.yml` memiliki tiga job. Job `backend-test` memakai PHP 8.4, ekstensi SQLite, dan Composer; job `frontend-build` memakai Node 22 dan `npm ci`; job `deploy` memakai Buildx, login ke GHCR, lalu mendorong dua image dengan tag `${{ github.sha }}`.

```
deploy:
  name: deploy
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  needs:
    - backend-test
    - frontend-build
  runs-on: ubuntu-latest
```

Kode 4.16: Kondisi job deploy

Nama image dihitung dari `GITHUB_REPOSITORY` dengan huruf kecil. Karena repository bernama `Ryzen0000001/uts-pl-24-535196-SV-24171`, nama image menjadi ghcr.io/ryzen0000001/uts-pl-24-535196-sv-24171-backend dan ghcr.io/ryzen0000001/uts-pl-24-535196-sv-24171-frontend. Tag SHA membuat setiap deploy dapat dihubungkan ke commit yang tepat.

```
echo "BACKEND_IMAGE=ghcr.io/${GITHUB_REPOSITORY,,}-backend" >> "$GITHUB_ENV"
echo "FRONTEND_IMAGE=ghcr.io/${GITHUB_REPOSITORY,,}-frontend" >> "$GITHUB_ENV"
```

Kode 4.17: Penentuan nama image GHCR

Rahasia yang disimpan di GitHub Secrets: `APP_KEY`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`, dan `DB_ROOT_PASSWORD`. Tidak ada nilai rahasia di dalam repository, Dockerfile, atau workflow.

4.7 Cara Menjalankan

Langkah reproduksi dari repositori:

```
git clone https://github.com/Ryzen0000001/uts-pl-24-535196-SV-24171.git
cd uts-pl-24-535196-SV-24171
```

```
cp .env.example .env
# isi APP_KEY, DB_PASSWORD, dan DB_ROOT_PASSWORD
docker compose up -d --build
docker compose ps
```

Kode 4.18: Menjalankan seluruh sistem dari nol

Setelah ketiga container sehat, frontend dapat dibuka di `http://localhost:8080` dan API di `http://localhost:8000/api`. Akun demo: `demo@example.com` / `password123`.

Bab 5

HASIL DAN PEMBAHASAN

5.1 Metode Pengujian

Pengujian tidak hanya memeriksa “berhasil” atau “gagal”. Setiap bukti diambil dari eksekusi nyata dan disimpan di direktori `laporan/`. Untuk API, status HTTP dan potongan body dicatat. Untuk container, keluaran `docker compose ps` dan log backend disimpan. Untuk pipeline, tautan run dan tangkapan layar halaman Actions diambil setelah run selesai.

5.2 CPMK 1: REST API, Autentikasi, dan CRUD

5.2.1 Hasil Tes Otomatis

Perintah `php artisan test` dijalankan di folder `backend`. Hasil terakhir: **16 tes lulus, 38 asersi**. Tes mencakup registrasi yang tidak tersedia, login valid dan tidak valid, logout yang mencabut token, akses tamu ke semua endpoint, kepemilikan data, validasi, serta CRUD.

```
PASS  Tests\Unit\ExampleTest
PASS  Tests\Feature\AuthTest
  OK register endpoint is not exposed
  OK login with invalid email returns 422
  OK login with valid credentials returns token
  OK login with wrong password returns 401
  OK logout revokes the token
  OK me without token returns 401
PASS  Tests\Feature\ExampleTest
PASS  Tests\Feature\ServiceLogTest
  OK guest gets 401 on every endpoint
  OK guest without json accept header still gets 401
  OK index lists only own logs
  OK store creates log and returns 201
  OK store with invalid data returns 422
  OK show returns own log but 404 for other users log
  OK update changes the log
  OK destroy deletes the log

Tests: 16 passed (38 assertions)
Duration: 0.45s
```

Kode 5.1: Keluaran `php artisan test` (ringkas)

Keluaran lengkap ada pada Lampiran A.1. Tes `guest without json accept header still gets 401` adalah tes yang ditambahkan setelah menemukan bug 500 pada proxy Nginx. Tanpa tes ini, bug tersebut hanya akan muncul saat diuji dari browser atau curl tanpa header JSON.

5.2.2 Verifikasi API Manual

Verifikasi dilakukan terhadap API yang berjalan di container, bukan server pengembangan. Tabel 5.1 merangkum hasilnya. Transkrip lengkap ada pada Lampiran A.5; token pada transkrip disamarkan.

Tabel 5.1: Hasil verifikasi status HTTP API

Method	Endpoint	Skenario	Status
POST	`/api/login`	Kredensial demo benar	200
POST	`/api/service-logs`	Payload valid	201
GET	`/api/service-logs`	Daftar milik pengguna	200
GET	`/api/service-logs/{id}`	Catatan milik sendiri	200
PUT	`/api/service-logs/{id}`	Ubah biaya	200
POST	`/api/service-logs`	`motor` kosong, `odometer` bukan angka, biaya negatif	422
GET	`/api/service-logs`	Tanpa header token	401
DELETE	`/api/service-logs/{id}`	Hapus catatan uji	200
POST	`/api/logout`	Cabut token	200

Contoh respons 422 dari pengujian:

```
{"message":"Motor wajib diisi. (and 3 more errors)",
 "errors":{
   "motor":["Motor wajib diisi."],
   "odometer_km":["Kilometer harus berupa angka bulat."],
   "service_type":["Jenis servis wajib diisi."],
   "cost":["Biaya tidak boleh kurang dari 0."]
 }}
```

Kode 5.2: Contoh respons validasi 422

5.2.3 Postman Collection

Berkas `postman/buku-servis.postman_collection.json` berisi tiga folder: Auth (Login, Me, Logout), Service Logs (List, Create, Create invalid, Show, Update, Delete), dan Tanpa token (List tanpa token). Variabel koleksi `baseUrl` dapat diubah ke `http://localhost:8000/api` atau `http://localhost:8080/api`. Skrip tes pada Login menyimpan token ke variabel ko-

leksi; skrip pada Create menyimpan `logId`; skrip pada Create invalid dan List tanpa token memeriksa status 422 dan 401.

Tangkapan layar `postman-login.png` belum tersedia saat laporan ini disusun. Permintaan dan status yang diuji tetap dicatat pada Tabel 3.4 dan transkrip Lampiran A.5.

Gambar 5.1: Respons login di Postman dan penyimpanan token

Tangkapan layar `postman-create-201.png` belum tersedia saat laporan ini disusun. Permintaan dan status yang diuji tetap dicatat pada Tabel 3.4 dan transkrip Lampiran A.5.

Gambar 5.2: Respons pembuatan catatan dengan status 201

Tangkapan layar `postman-422.png` belum tersedia saat laporan ini disusun. Permintaan dan status yang diuji tetap dicatat pada Tabel 3.4 dan transkrip Lampiran A.5.

Gambar 5.3: Respons validasi dengan status 422

Tangkapan layar `postman-401.png` belum tersedia saat laporan ini disusun. Permintaan dan status yang diuji tetap dicatat pada Tabel 3.4 dan transkrip Lampiran A.5.

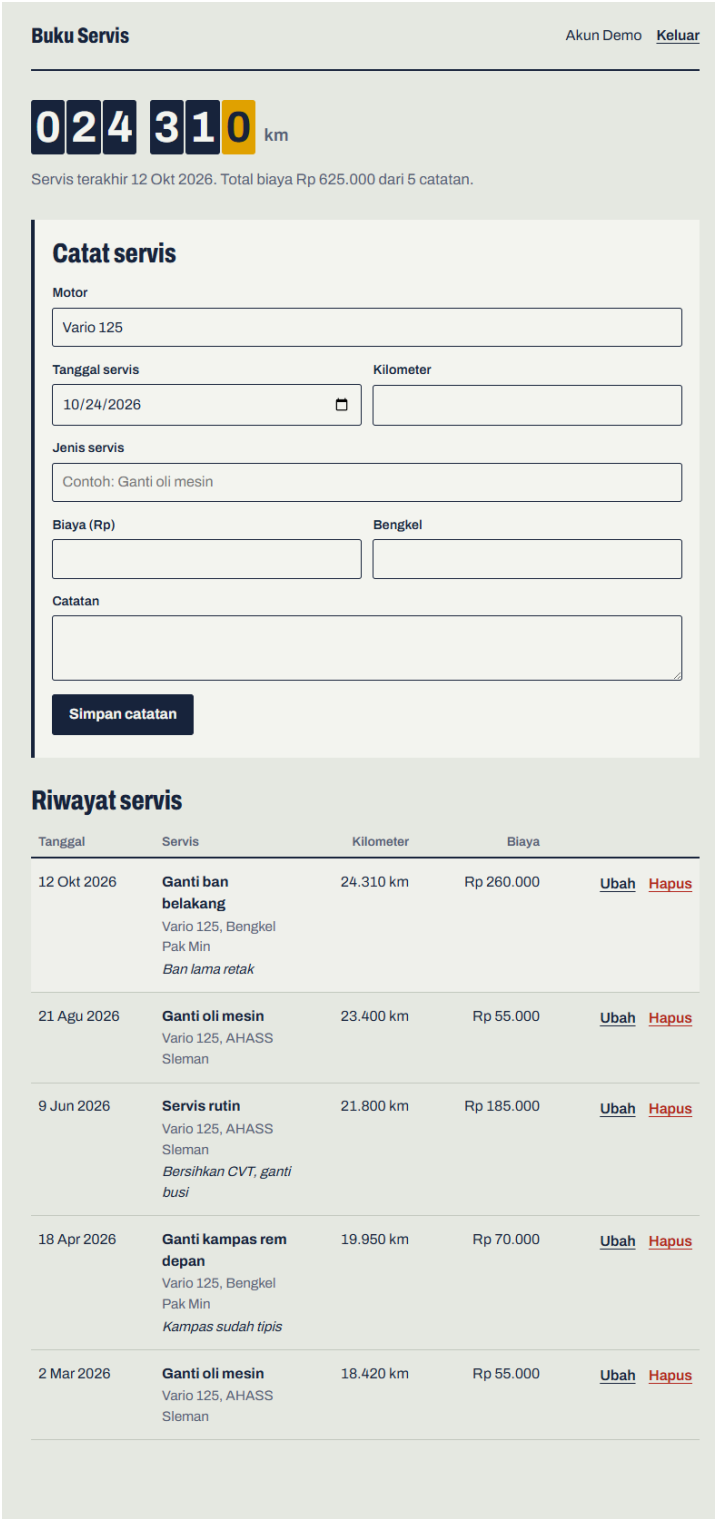
Gambar 5.4: Respons tanpa token dengan status 401

5.3 CPMK 2: Frontend Vue dan Integrasi Axios

Antarmuka diuji dengan browser pada alamat `http://localhost:8080`. Alur yang diuji: masuk dengan akun demo, melihat daftar, menambah catatan, mengubah catatan, menghapus catatan, lalu keluar. Tangkapan layar diambil dari aplikasi yang berjalan di container, bukan mockup.

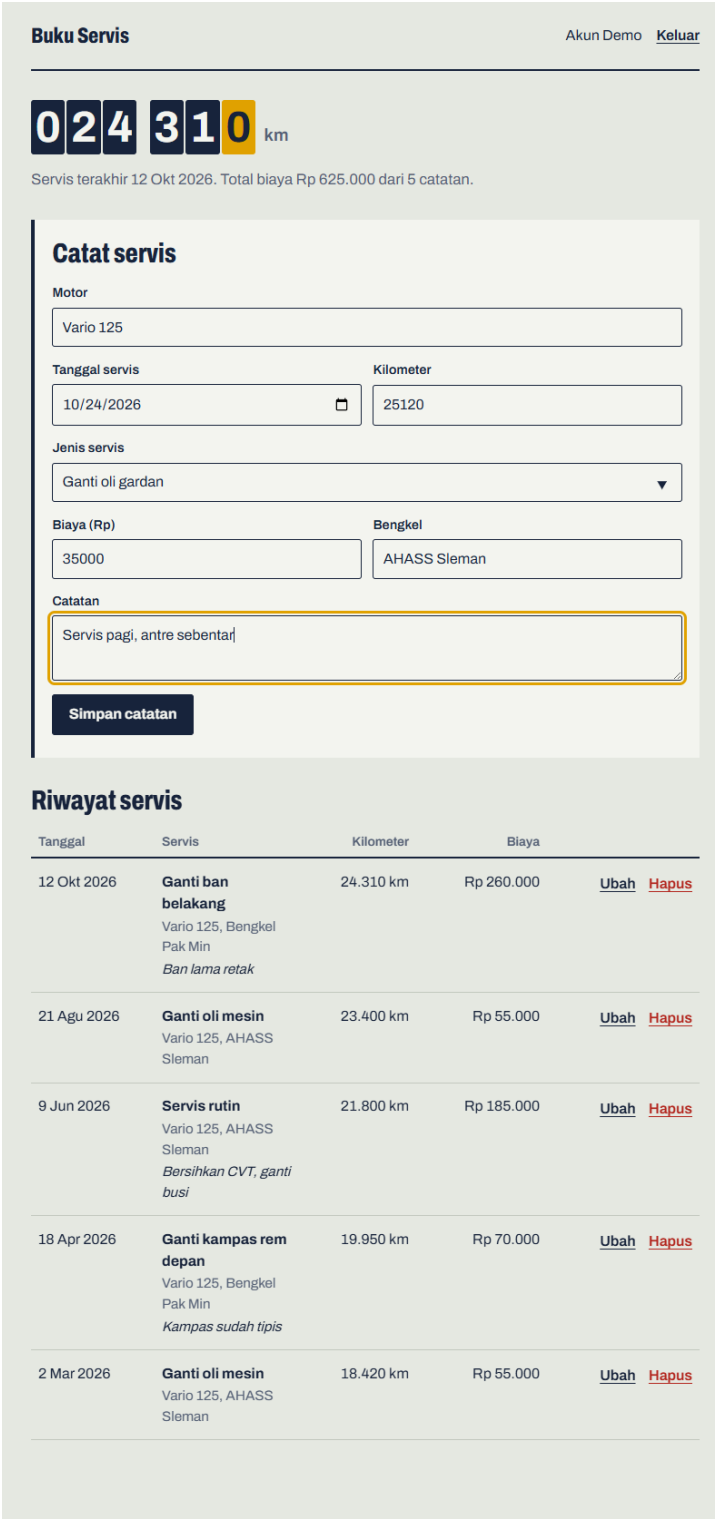


Gambar 5.5: Halaman masuk sebelum autentikasi

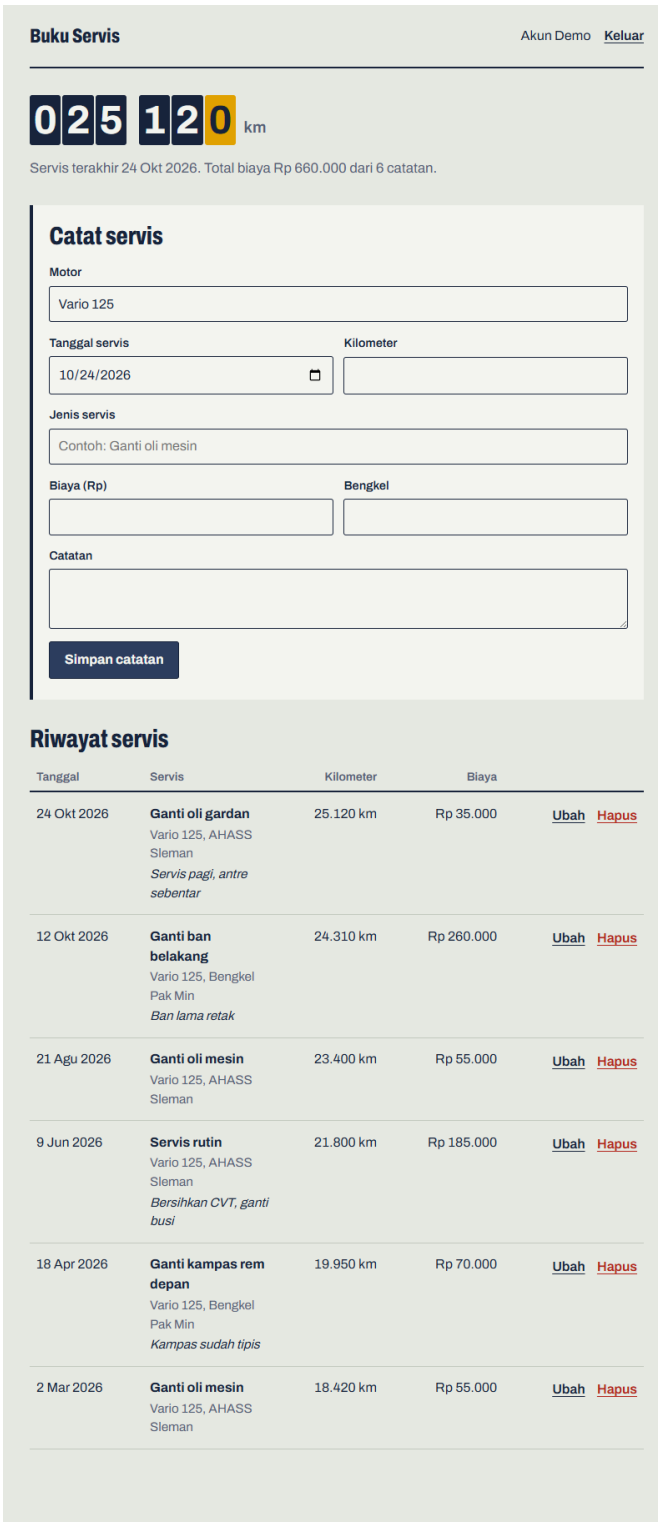


Gambar 5.6: Halaman daftar catatan setelah login

Formulir tambah diisi dengan data uji: motor Vario 125, tanggal 24 Oktober 2026, odometer 25.120 km, jenis servis “Ganti oli gardan”, biaya Rp35.000, bengkel AHASS Sleman, dan catatan “Servis pagi, antre sebentar”. Sebelum disimpan, ringkasan menunjukkan servis terakhir 12 Oktober 2026 dengan total Rp625.000 dari lima catatan. Setelah disimpan, catatan muncul di bagian atas daftar karena diurutkan berdasarkan tanggal terbaru; ringkasan berubah menjadi Rp660.000 dari enam catatan.

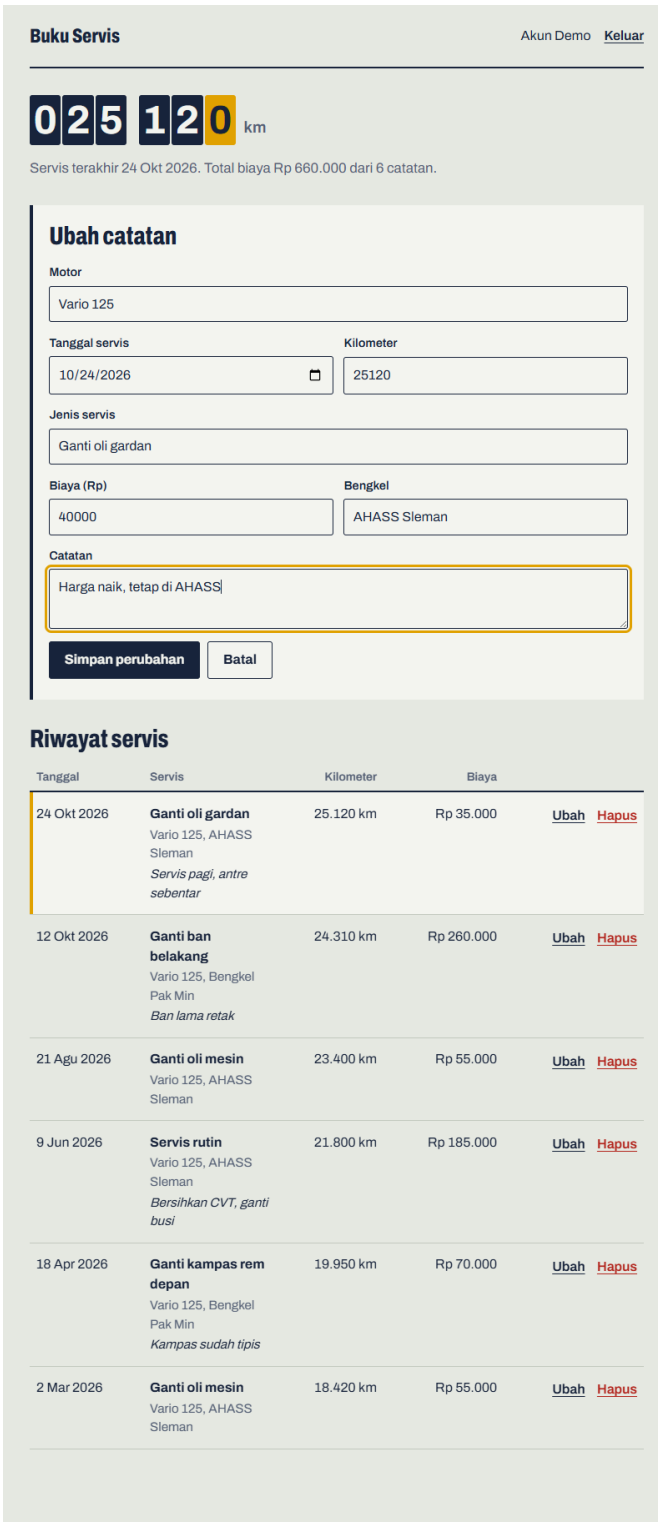


Gambar 5.7: Formulir tambah catatan yang sudah diisi

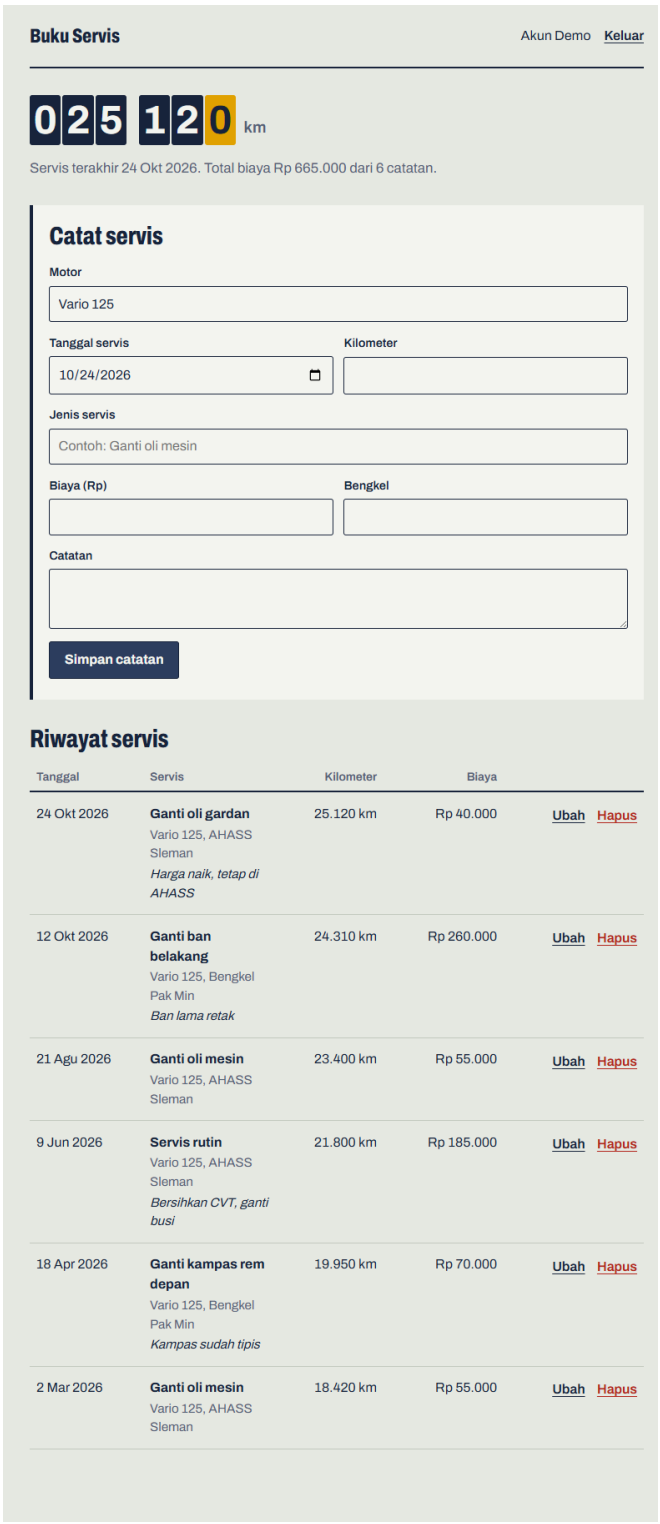


Gambar 5.8: Catatan baru muncul di daftar setelah disimpan

Untuk mengubah, tombol “Ubah” pada baris catatan mengisi formulir dan mengubah judul menjadi “Ubah catatan”. Biaya diubah dari Rp35.000 menjadi Rp40.000 dan catatan diganti menjadi “Harga naik, tetap di AHASS”. Setelah disimpan, daftar dimuat ulang dan total biaya pada ringkasan menjadi Rp665.000.

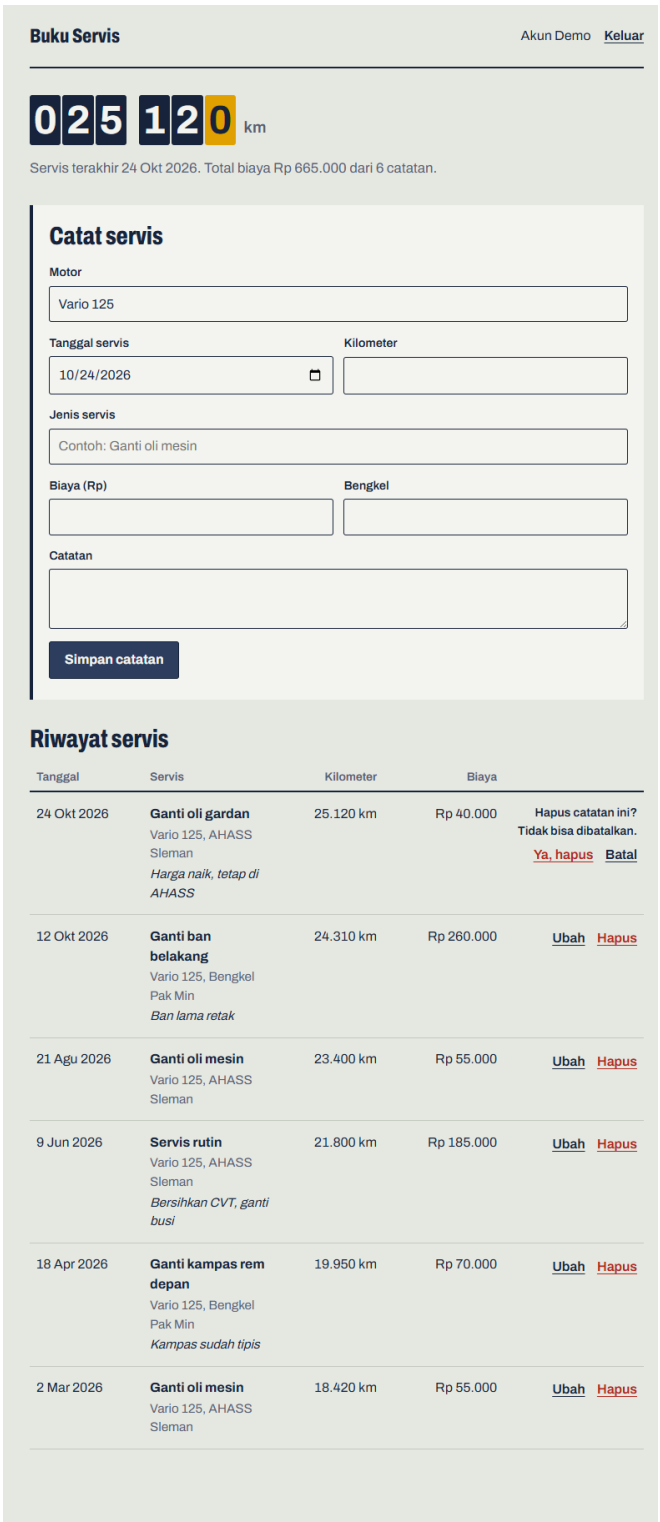


Gambar 5.9: Formulir ubah dengan data catatan terpilih

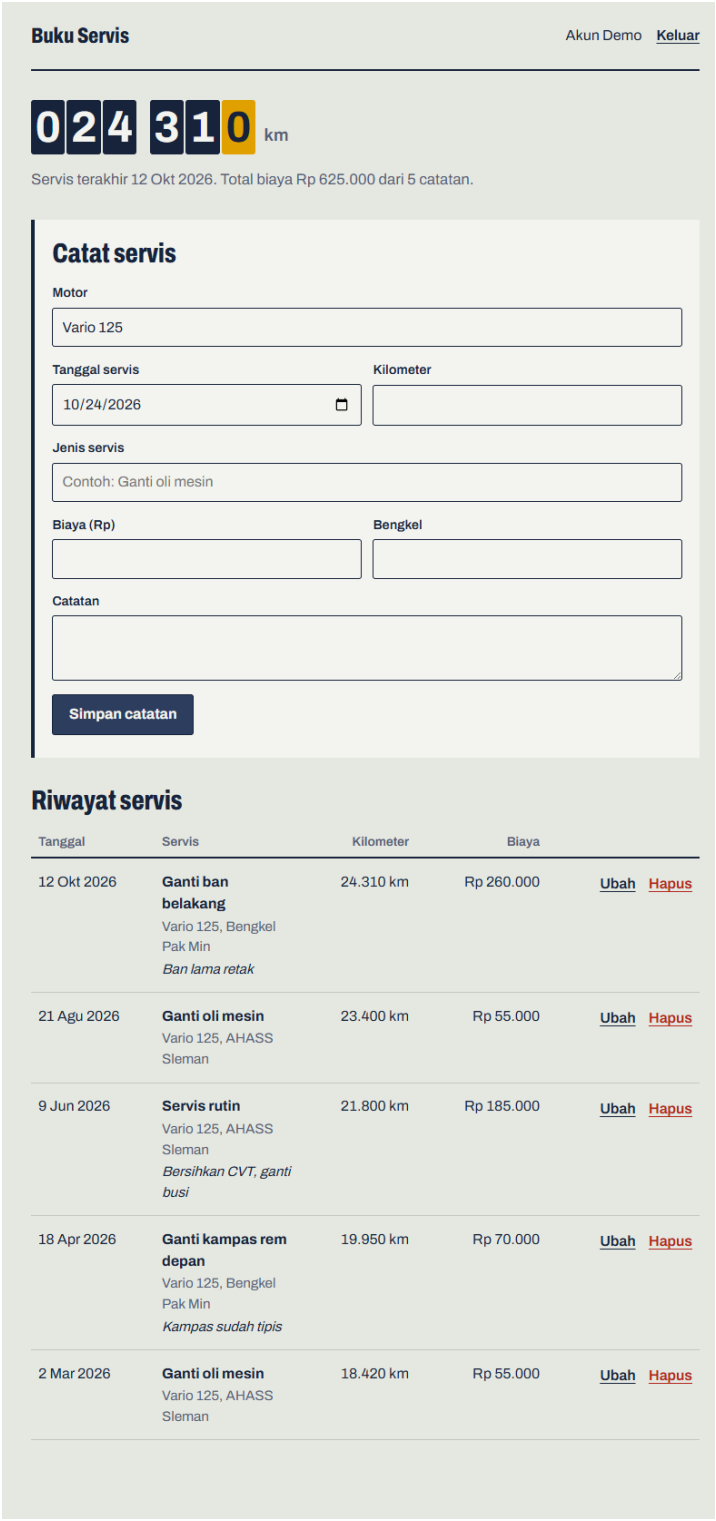


Gambar 5.10: Daftar setelah biaya diubah

Hapus memakai konfirmasi di dalam baris. Pengguna harus menekan “Ya, hapus”; tombol “Batal” menutup konfirmasi tanpa mengirim permintaan. Setelah catatan Ganti oli gardan dihapus, daftar kembali berisi lima catatan dengan total Rp625.000 dan pesan “Catatan dihapus.” muncul.



Gambar 5.11: Konfirmasi hapus dua langkah



Gambar 5.12: Daftar setelah catatan dihapus

Dari sisi integrasi, frontend memanggil `/api/...` pada origin yang sama (`localhost:8080`). Nginx meneruskan ke backend. Karena token dikirim pada header `Authorization`, tidak ada cookie lintas origin dan tidak ada konfigurasi CORS khusus di frontend. Jika token dicabut, interceptor 401 memicu kembali ke halaman masuk.

5.4 CPMK 3: Docker Compose, FrankenPHP, dan Persistensi

5.4.1 Status Container

Setelah `docker compose up -d -build`, ketiga container berjalan dan sehat. Tabel 5.2 merangkum status dari `docker compose ps`; keluaran mentahnya ada pada Lampiran A.2.

Tabel 5.2: Status container dari `docker compose ps`

Container	Service	Image	Status dan port
`uts-pl-backend-1`	backend	`uts-pl-backend:local`	Up (healthy), `0.0.0.0:8000->8000/tcp`
`uts-pl-db-1`	db	`mysql:8.4.13`	Up (healthy), port internal 3306
`uts-pl-frontend-1`	frontend	`uts-pl-frontend:local`	Up (healthy), `0.0.0.0:8080->80/tcp`

5.4.2 Bukti FrankenPHP

Log backend memuat baris startup FrankenPHP dan versi PHP 8.4.27. Access log menunjukkan permintaan `POST /api/login`, `POST /api/service-logs`, `GET /api/service-logs`, `PUT`, `DELETE`, dan respons 401 untuk permintaan tanpa token. Header respons juga memuat `Server: FrankenPHP Caddy`. Ini membuktikan backend tidak dijalankan dengan `php artisan serve` atau PHP-FPM.

```
STARTUP: {"logger":"frankenphp","msg":"FrankenPHP started","php_version":"8.4.27",
          "total_threads":48,"worker_threads":0,"max_threads":48}
POST /api/login -> 200
POST /api/service-logs -> 201
GET /api/service-logs -> 200
GET /api/service-logs/7 -> 200
PUT /api/service-logs/7 -> 200
POST /api/service-logs -> 422
GET /api/service-logs -> 401
DELETE /api/service-logs/7 -> 200
POST /api/logout -> 200
```

Kode 5.3: Potongan bukti log FrankenPHP

Pemeriksaan proses di dalam container juga menunjukkan perintah `frankenphp run -config /etc/caddy/Caddyfile` dan user `app`, bukan root. Image backend berukuran sekitar 374 MB; image frontend sekitar 92,9 MB. Image frontend tidak memuat `node_modules`, dan image backend tidak memuat Composer.

5.4.3 Persistensi Volume

Uji persistensi dilakukan dengan membuat catatan uji, menghentikan seluruh container tanpa menghapus volume, lalu menyalakannya kembali. Catatan dengan id 8 masih ada setelah `docker compose up -d`. Setelah verifikasi, catatan uji dihapus agar data demo tetap rapi.

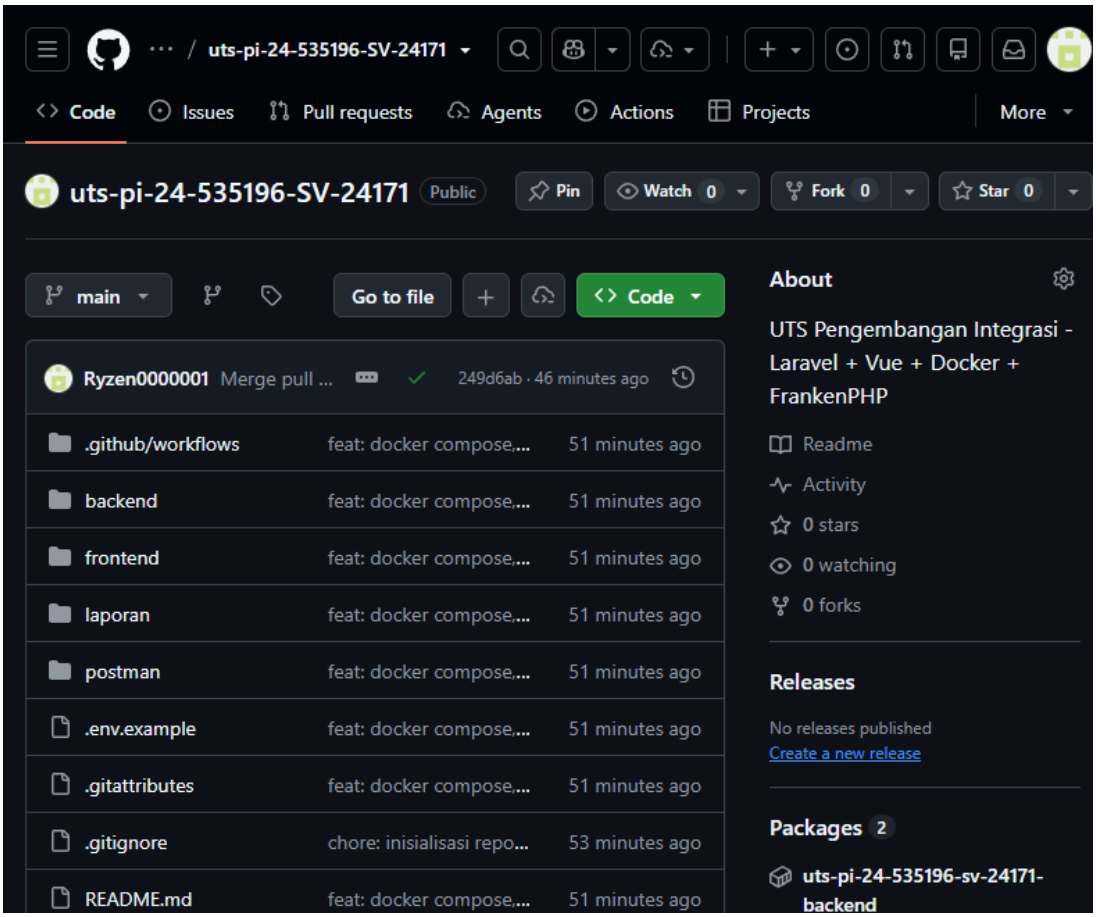
```
SEBELUM DOWN: id uji = 8, jumlah catatan = 6
docker compose down
docker compose up -d
backend /up -> 200
SESUDAH UP: jumlah catatan = 6; data id=8 ditemukan = True;
              service_type = Uji persistensi volume
Pembersihan: DELETE id=8 -> {"message":"Catatan dihapus."}
```

Kode 5.4: Hasil uji persistensi

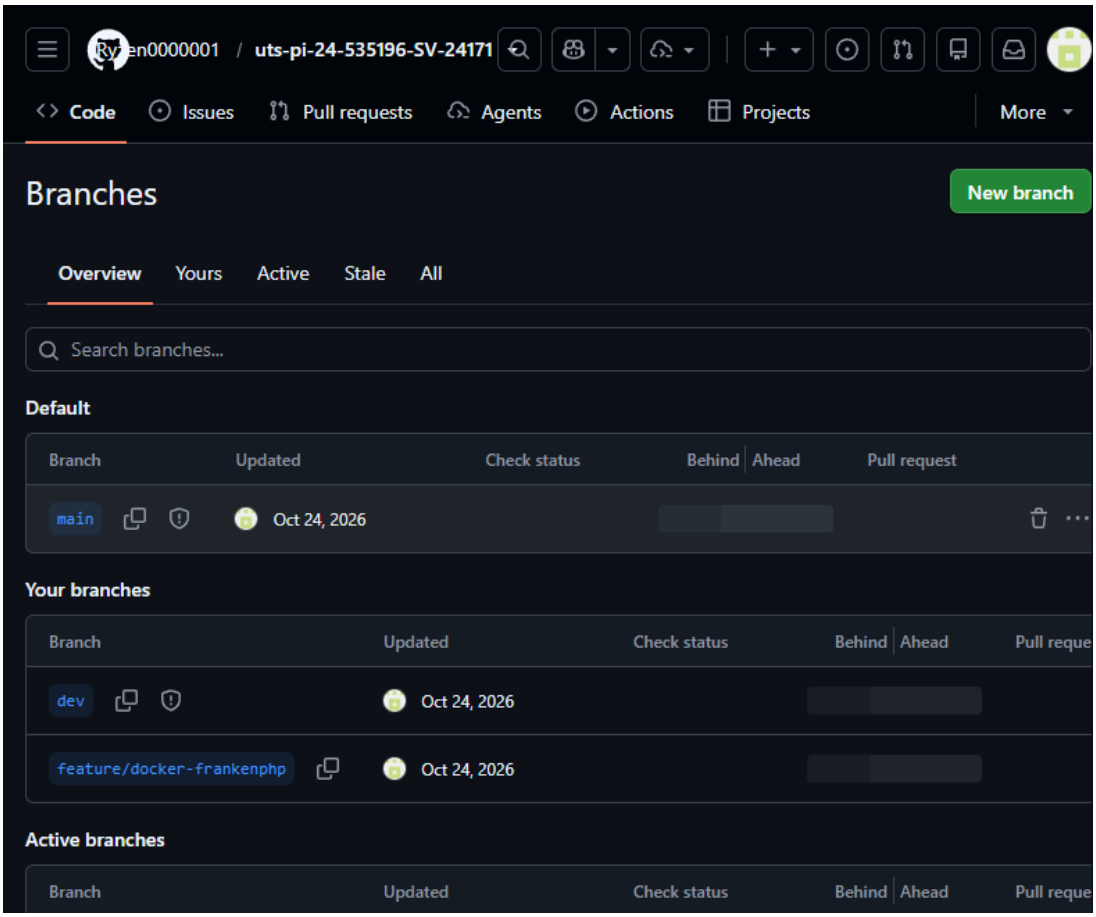
5.5 CPMK 4: Git, Branch Protection, dan CI/CD

5.5.1 Repositori dan Branch

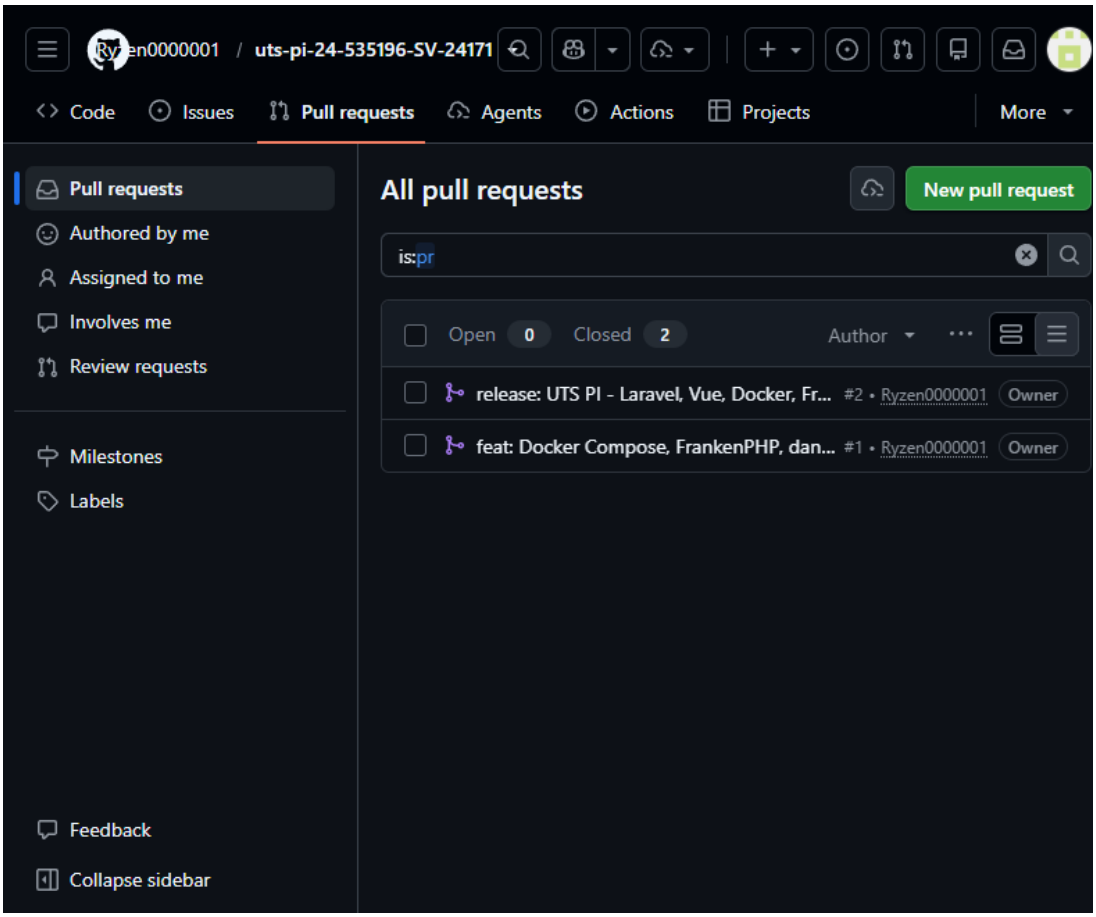
Repositori publik berada di https://github.com/Ryzen0000001/uts-pl-24-535196-SV-24171. Branch yang ada: `main`, `dev`, dan `feature/docker-frankenphp`. Pekerjaan fitur masuk ke `dev` melalui PR #1, lalu `dev` masuk ke `main` melalui PR #2. Kedua PR sudah digabung dan branch fitur tetap disimpan sebagai jejak pekerjaan.



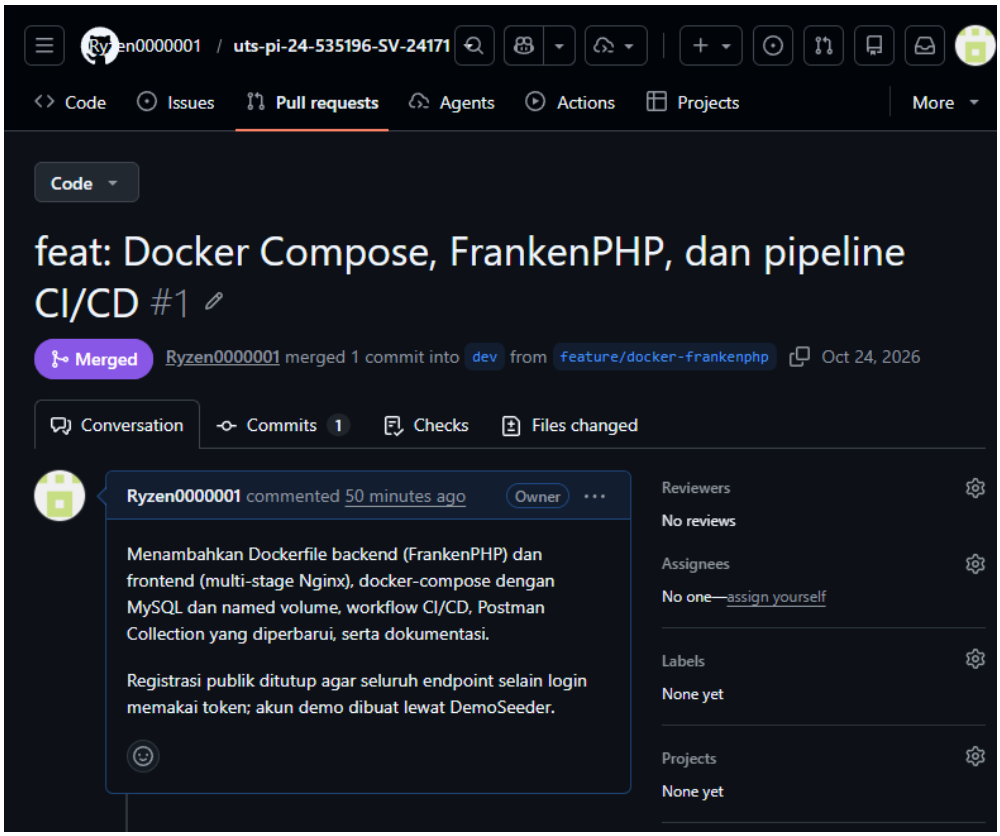
Gambar 5.13: Halaman utama repositori publik



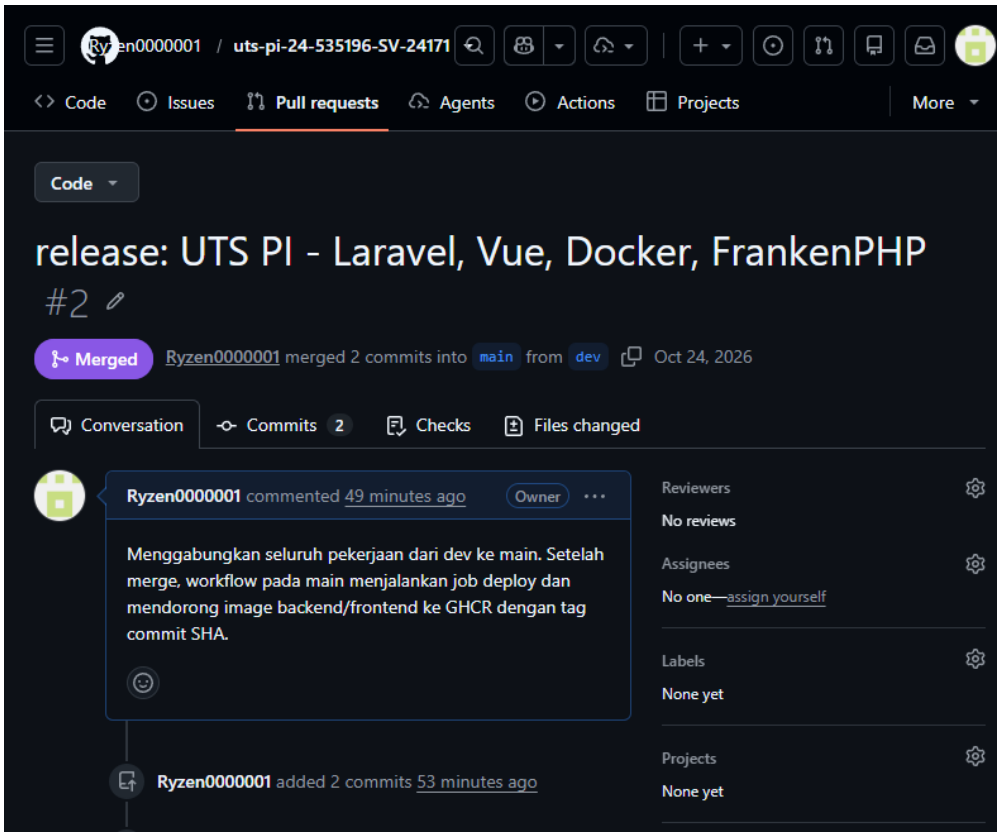
Gambar 5.14: Daftar branch `main`, `dev`, dan fitur



Gambar 5.15: Dua pull request yang sudah digabung



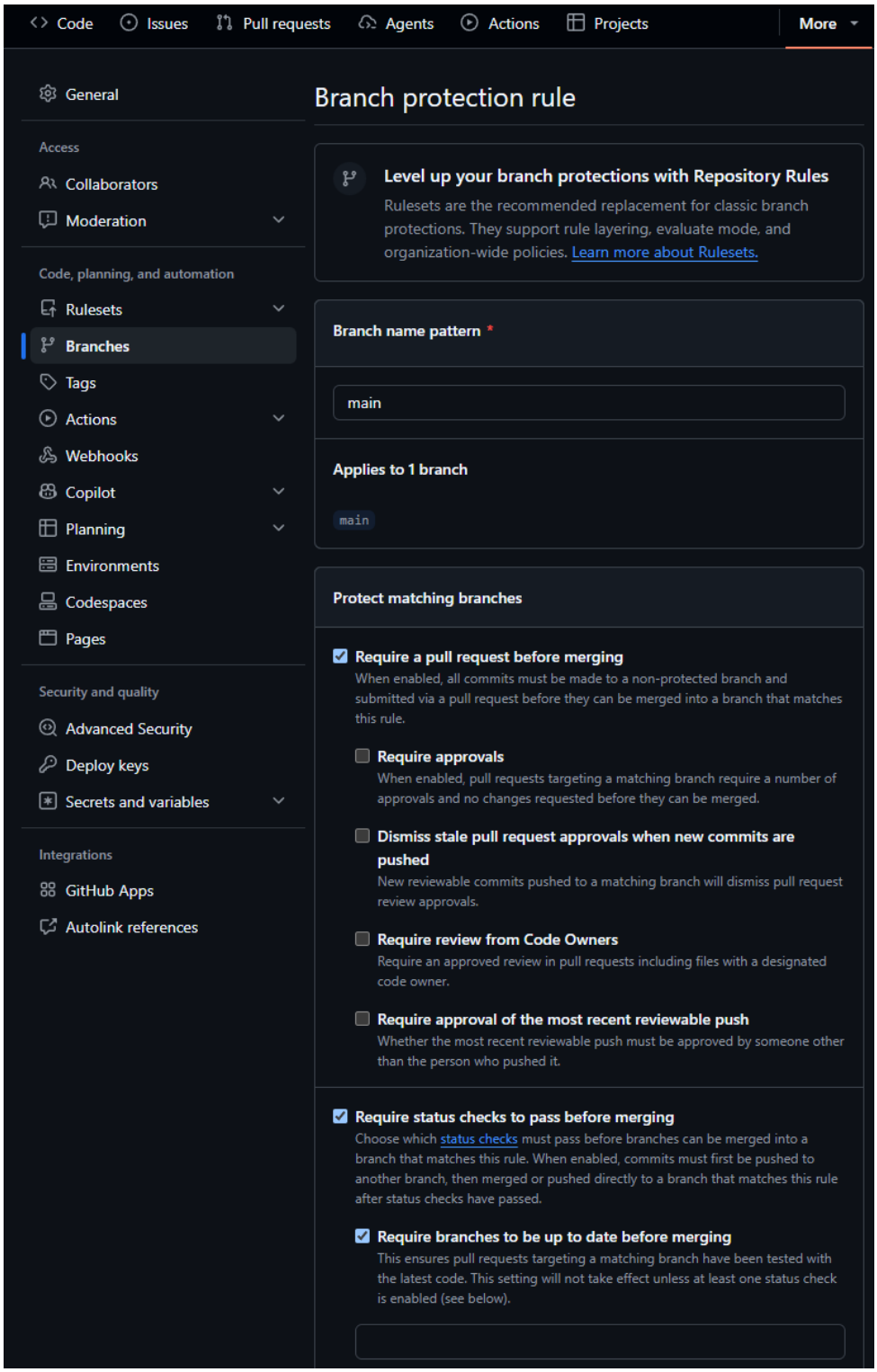
Gambar 5.16: PR #1: fitur Docker dan FrankenPHP masuk ke `dev`



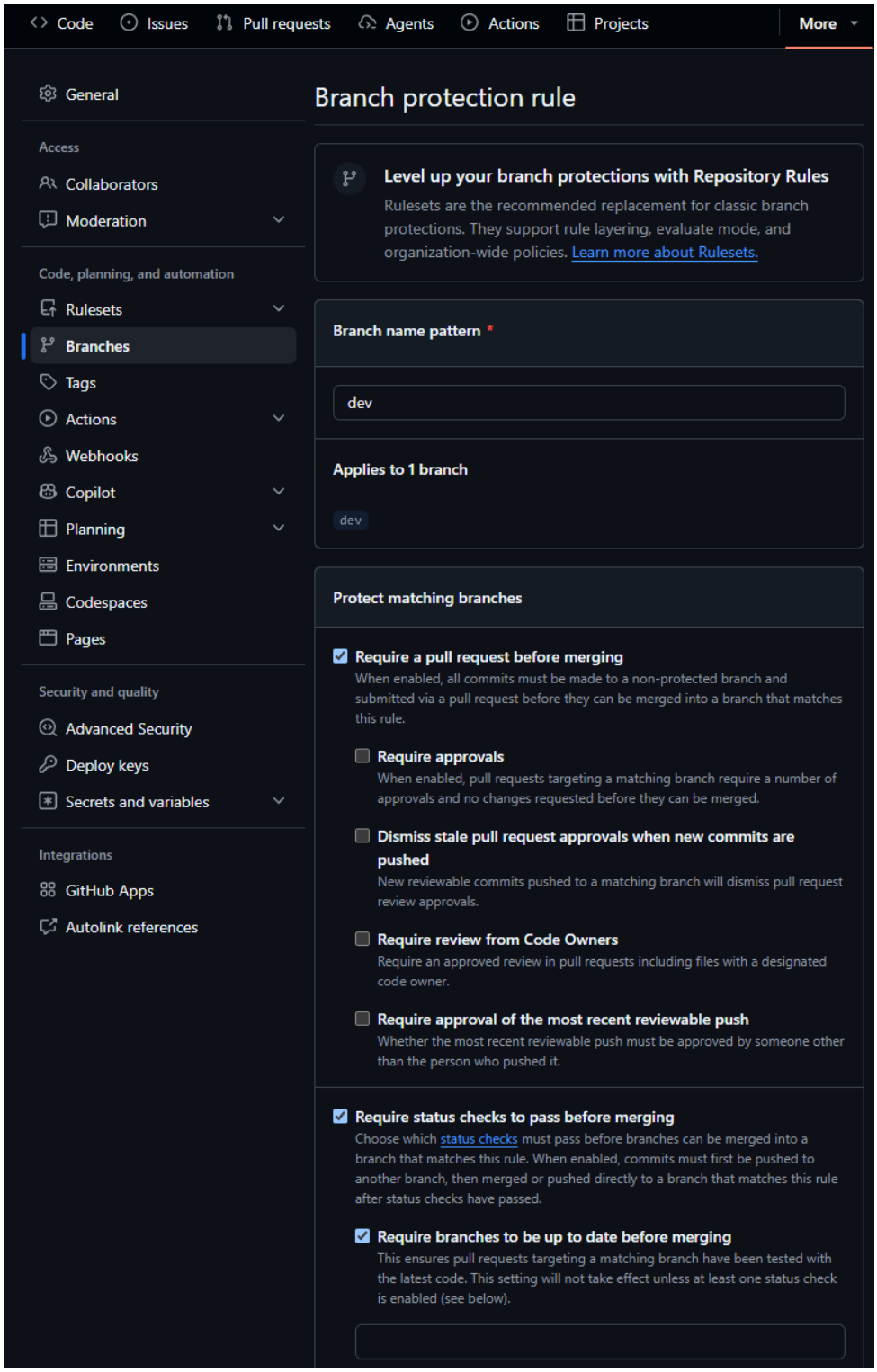
Gambar 5.17: PR #2: `dev` masuk ke `main`

5.5.2 Branch Protection

Aturan pada `main` dan `dev` mewajibkan pull request sebelum merge, status check `backend-test` dan `frontend-build`, serta branch yang up-to-date. Opsi "Do not allow bypassing the above settings" diaktifkan sehingga aturan juga berlaku untuk administrator. Force push dan penghapusan branch dinonaktifkan.



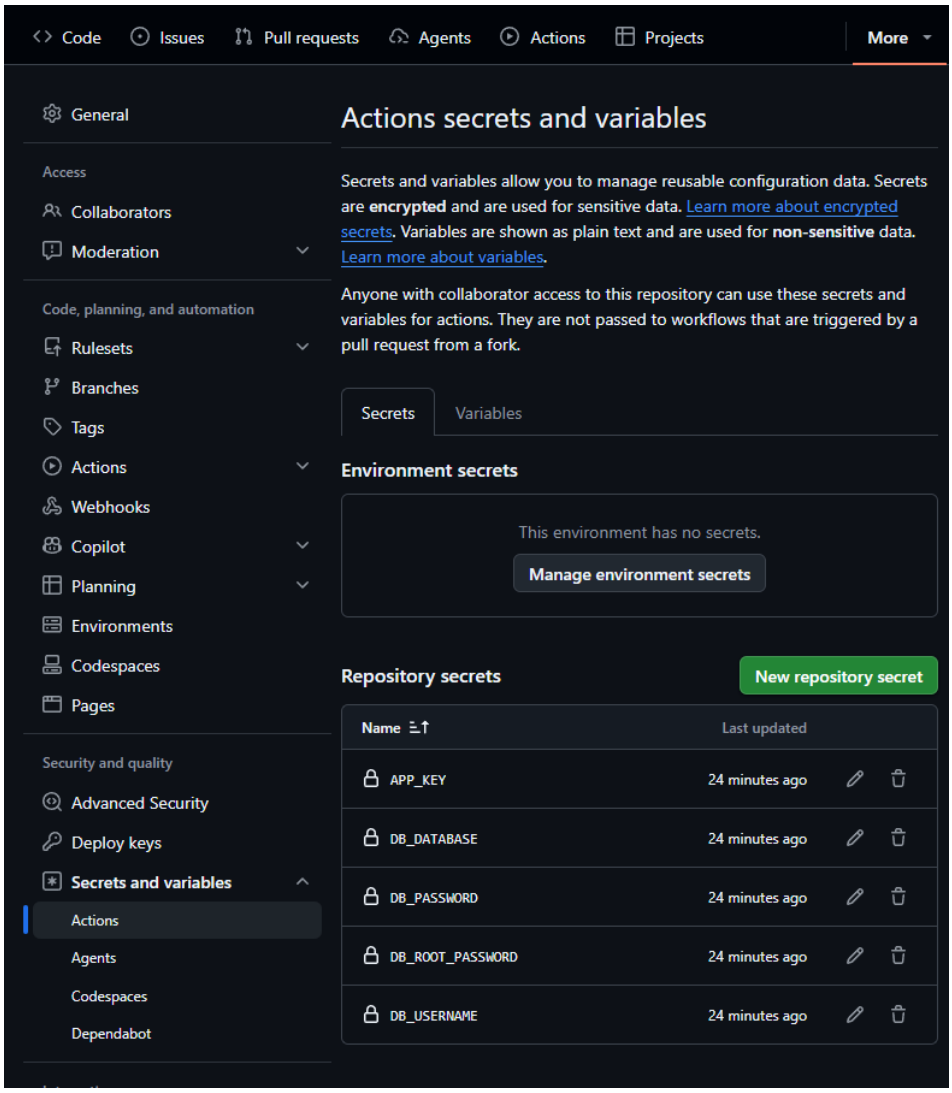
Gambar 5.18: Branch protection pada `main`



Gambar 5.19: Branch protection pada `dev`

5.5.3 GitHub Secrets

Lima rahasia disimpan di `Settings > Secrets and variables > Actions`: `APP_KEY`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`, dan `DB_ROOT_PASSWORD`. Halaman secrets hanya menampilkan nama, bukan nilai.



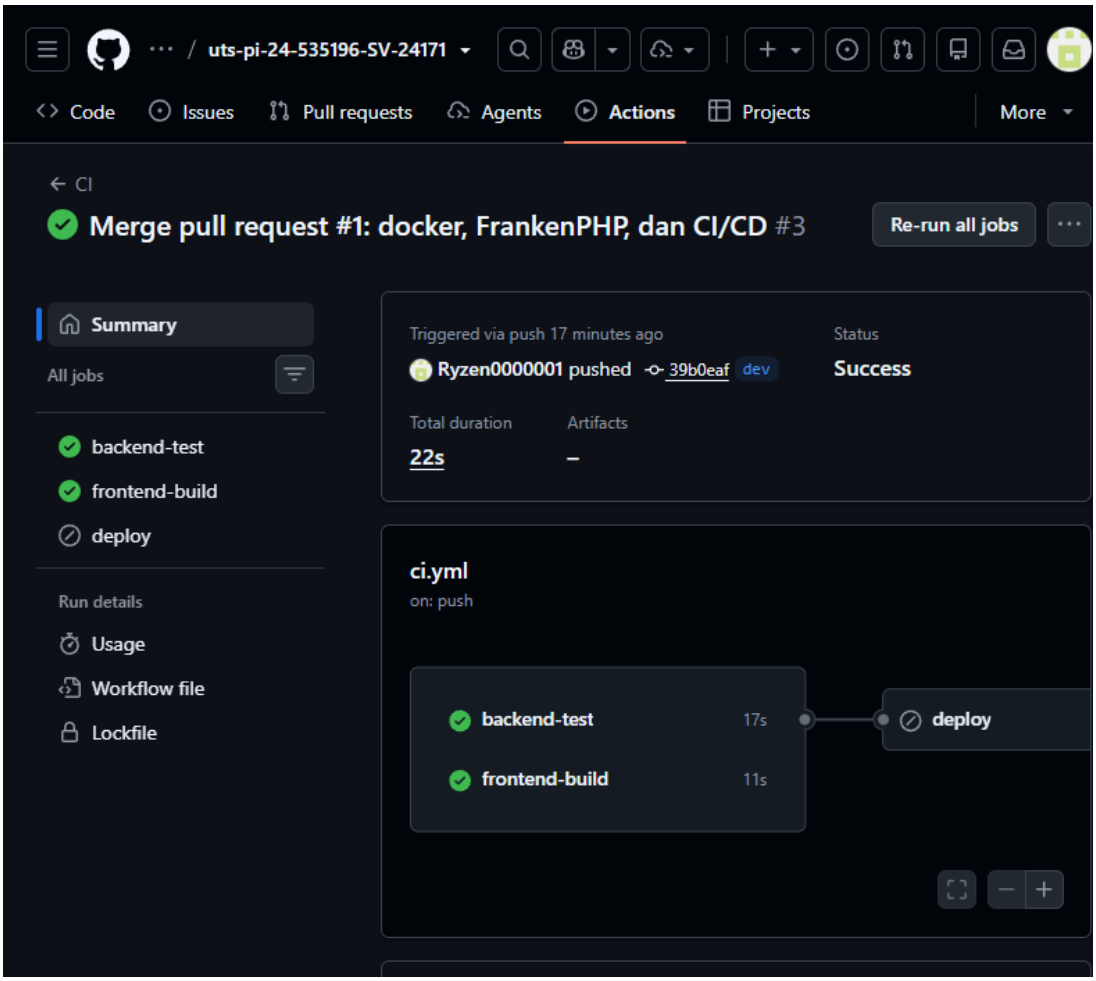
Gambar 5.20: Daftar GitHub Secrets tanpa nilai rahasia

5.5.4 Hasil Pipeline

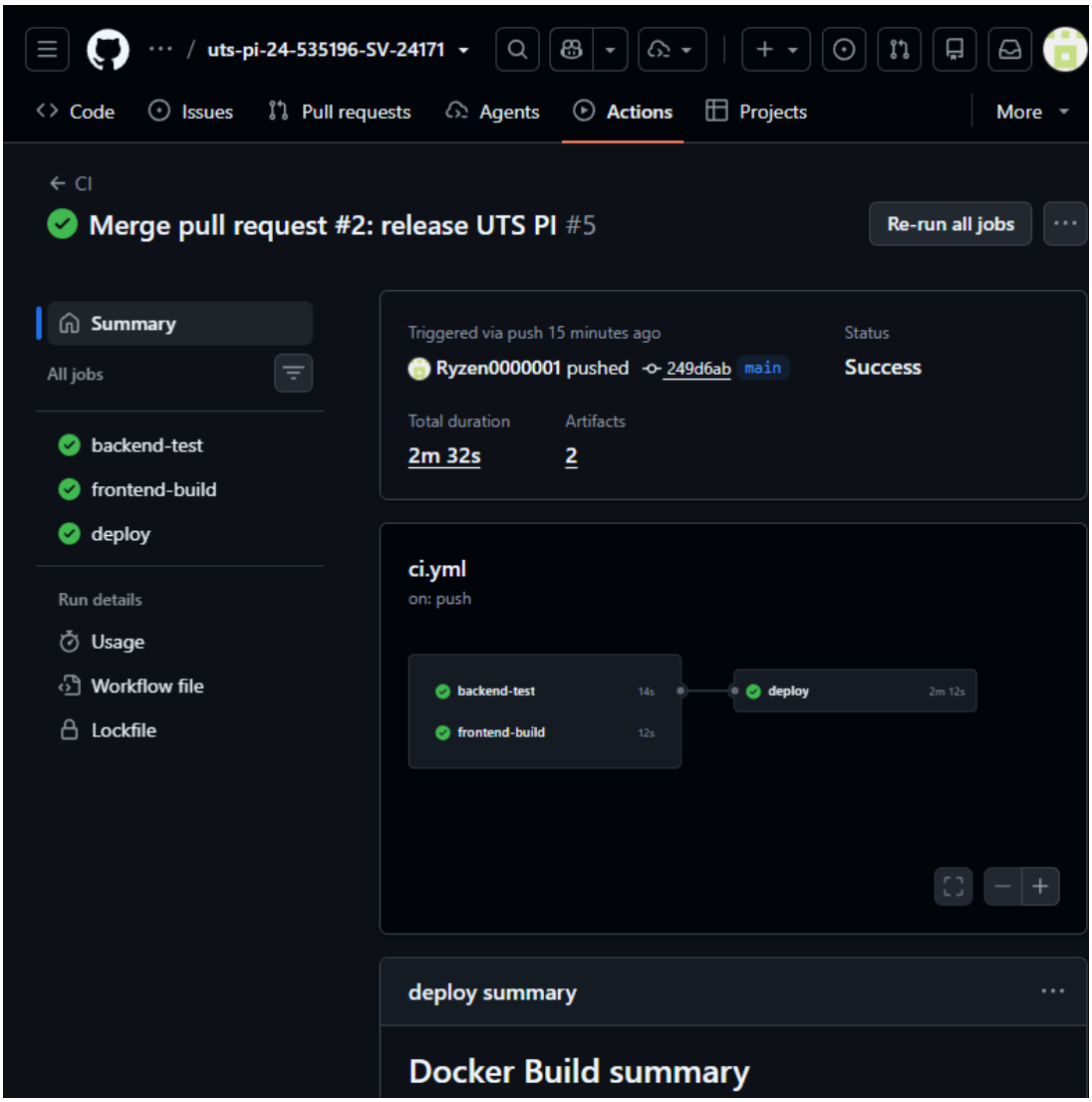
Push ke branch fitur dan `dev` menjalankan dua job pengujian tanpa deploy. Push ke `main` menjalankan ketiga job; job deploy membangun dan mendorong image ke GHCR. Tabel 5.3 merangkum run yang diamati.

Tabel 5.3: Run GitHub Actions yang diamati

Event	Commit/Run	Job	Deploy
Push fitur	CI pada branch `feature/docker-frankenphp`	test + build	Tidak
PR #1	Pemeriksaan sebelum merge ke `dev`	test + build	Tidak
Push `dev`	Setelah PR #1 digabung	test + build	Tidak
PR #2	Pemeriksaan sebelum merge ke `main`	test + build	Tidak
Push `main`	Setelah PR #2 digabung	test + build + deploy	Ya



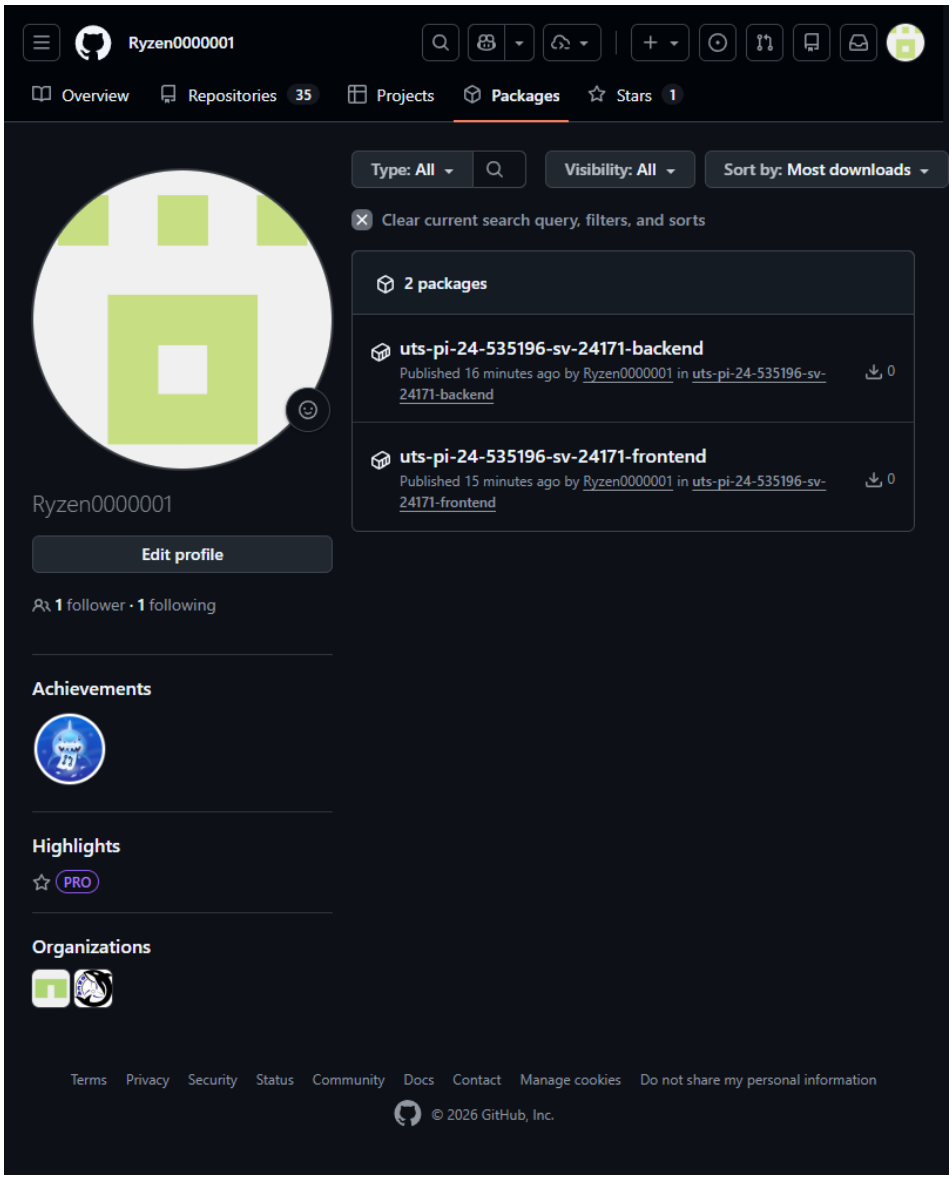
Gambar 5.21: Run pada `dev`: dua job lulus, deploy tidak muncul



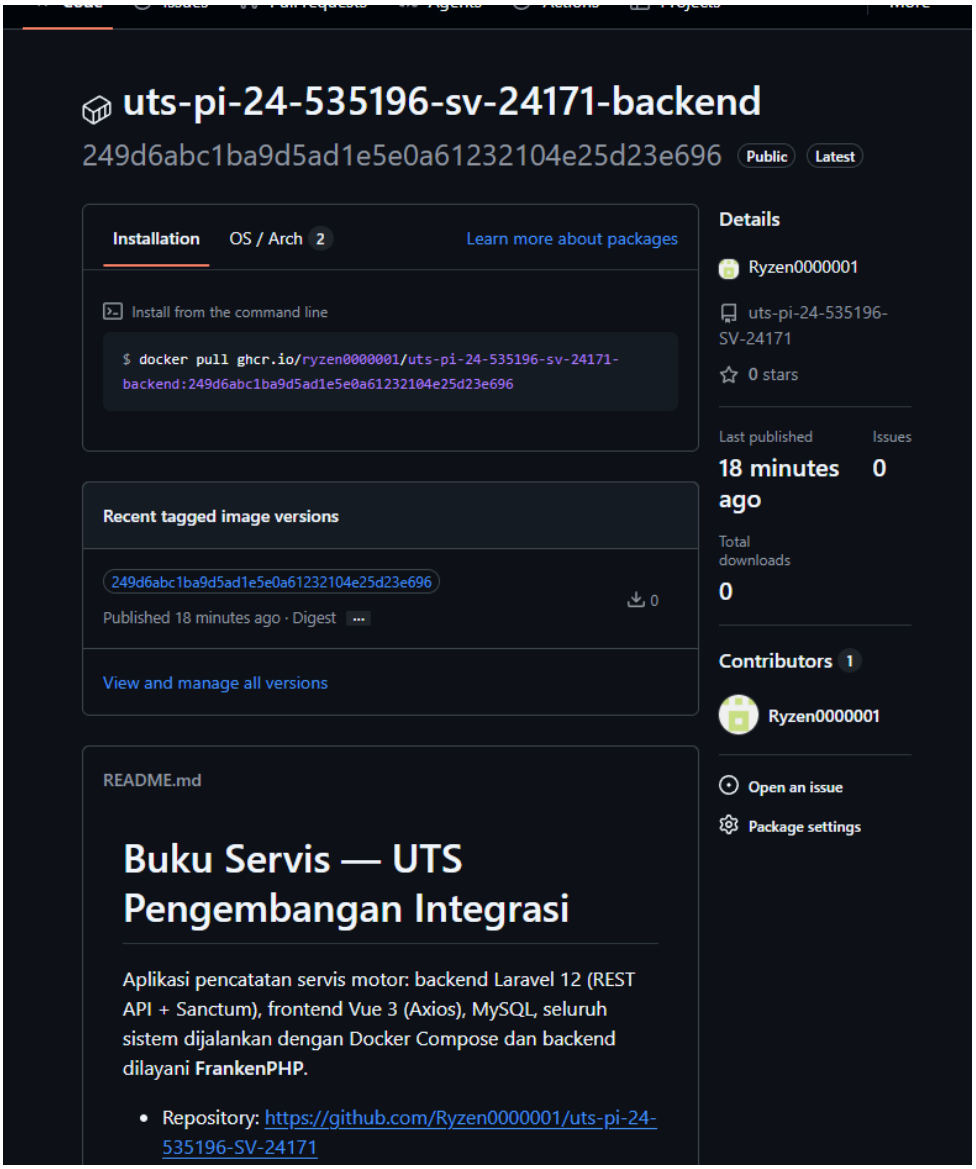
Gambar 5.22: Run pada `main`: backend-test, frontend-build, dan deploy lulus

5.5.5 Image di GHCR

Dua package container dipublikasikan: backend dan frontend. Halaman package menampilkan tag commit SHA dan tautan ke repositori. Setelah repo berganti nama menjadi `uts-pl`, nama package mengikuti nama repositori baru.



Gambar 5.23: Daftar package container di GHCR



Gambar 5.24: Halaman package backend dan tag SHA

5.6 CPMK 5: Dokumentasi dan Reproduksibilitas

Dokumentasi terdiri dari README, Postman Collection, komentar pada konfigurasi non-obvious, dan laporan ini. README memuat cara menjalankan, tabel endpoint, perintah pengujian, penjelasan alamat API, serta daftar secrets. Postman Collection memungkinkan penguji mencoba endpoint tanpa menulis ulang body JSON.

Yang belum sepenuhnya otomatis adalah pengambilan tangkapan layar Postman dan pengujian UI. Keduanya masih manual karena memerlukan aplikasi desktop dan pemeriksaan visual. Untuk pengembangan lanjutan, UI test dapat memakai Playwright, sedangkan API test dapat dijalankan dengan Newman di pipeline.

5.7 Pembahasan Alur Data End-to-End

Saat pengguna mengisi formulir dan menekan “Simpan catatan”, Vue membentuk objek payload. `ServiceForm` memanggil `api.post('/service-logs', payload)`. Axios menambahkan `baseURL` dari `VITE_API_URL` dan header `Authorization` dari token di `localStorage`. Di Docker, permintaan menuju `http://localhost:8080/api/service-logs`. Nginx mencocokkan `location /api/` dan meneruskannya ke `http://backend:8000/api/service-logs`.

Di backend, route Laravel mencocokkan `POST /api/service-logs`. Middleware `auth:sanctum` memeriksa token. Jika valid, `ServiceLogRequest` menjalankan validasi. Jika lolos, controller memanggil relasi `$request->user()->serviceLogs()->create($request->validated())`. Eloquent mengisi `user_id` dari pengguna yang terautentikasi dan menjalankan `INSERT` ke MySQL. Controller mengembalikan JSON dengan status 201.

Frontend menerima respons, mengosongkan formulir, memuat ulang daftar dengan `GET /api/service-logs`, dan menampilkan pesan “Catatan disimpan.”. Jika validasi gagal, respons 422 berisi `errors`; formulir menampilkan pesan di bawah field yang salah. Jika token hilang, respons 401 memicu event `auth:expired` dan router mengarahkan pengguna ke `/masuk`.

5.8 Kendala dan Solusi

Tabel 5.4: Kendala yang ditemui dan solusinya

No	Kendala	Solusi
1	Frontend sempat memanggil `localhost:8000` dari browser, sehingga alamat API berbeda antara lokal dan Docker.	Alamat dipindah ke `VITE_API_URL`; Docker memakai `/api` dan Nginx mem-proxy ke `backend:8000`.
2	Nginx dengan `proxy_pass http://backend:8000` dapat menyimpan DNS lama saat container backend dibuat ulang.	Memakai `resolver 127.0.0.11` dan variabel `$backend` agar resolusi dilakukan ulang.
3	Backend sempat 500 untuk permintaan tanpa `Accept: application/json` karena middleware mencari route `login`.	Menambahkan `redirectGuestsTo(fn () => null)` dan tes regresi 401.
4	Pesan validasi `min` untuk biaya masih berbahasa Inggris.	Menambahkan pesan per atribut `cost.min` dan `odometer_km.min`; pesan size Laravel tidak dibaca dari larik biasa.
5	Container backend mulai sebelum MySQL siap.	Entrypoint menunggu dengan 30 percobaan, lalu migrasi dan seeder.
6	Registrasi publik tidak boleh ada, tetapi penguji membutuhkan akun.	`DemoSeeder` membuat akun demo saat backend mulai.
7	Data hilang jika volume ikut dihapus.	Volume bernama `uts-pl_db_data`; dokumentasi menjelaskan perbedaan `down` dan `down -v`.
8	Deploy bisa terjadi dari branch selain `main`.	Job deploy diberi kondisi `github.ref== 'refs/heads/main'`; branch protection mencegah push langsung.
9	Rahasia berisiko masuk repositori.	`.env` di-`.gitignore`; nilai produksi di GitHub Secrets; Dockerfile tidak memuat secret.
10	Nama repositori awal memakai akhiran `uts-pi`, tidak sesuai kode mata kuliah.	Repositori, nama image, volume, README, dan laporan diseragamkan menjadi `uts-pl`.

Kendala nomor 3 adalah yang paling penting untuk dicatat. Tanpa pengujian dari proxy, bug ini tidak terlihat karena `getJson()` selalu mengirim header JSON. Perbaikannya kecil, tetapi dampaknya besar: API sekarang konsisten mengembalikan 401 JSON untuk semua permintaan tanpa token.

5.9 Kesesuaian dengan Ketentuan

Tabel 5.5: Pemetaan ketentuan tugas ke bukti

Ketentuan	Bukti	Lokasi
REST API Laravel + Sanctum	`routes/api.php`, `AuthController`, tes 401	Subbab 4.3, 5.2
Semua endpoint selain login memakai token	Grup `auth:sanctum`; tes tamu 401	Kode 4.2, Tabel 5.1
CRUD status 201/422/401	Tes dan transkrip curl	Tabel 5.1
Minimal 2 feature test	16 tes, 38 asersi	Lampiran A.1
Postman Collection	`postman/buku-servis.postman_collection.json`	Subbab 5.2
Vue + Axios + token	`api.js`, `auth.js`, halaman UI	Subbab 4.4, 5.3
API URL dari env	`VITE_API_URL`	Kode 4.8
Docker Compose 3 service	db, backend, frontend	Tabel 3.5
Jaringan service-name	`DB_HOST=db`, proxy `backend:8000`	Kode 4.14
Volume permanen	`uts-pl_db_data`; uji id 8	Subbab 5.5
Kredensial dari `.env`	`.env.example`, secrets	Subbab 4.7, 5.4
FrankenPHP (bonus)	Dockerfile, Caddyfile, log startup	Subbab 4.5, 5.5
CI/CD	`ci.yml`; job test, build, deploy	Tabel 5.3
Deploy hanya dari `main`	Kondisi job deploy	Kode 4.16
Image GHCR tag SHA	Halaman package	Gambar 5.5.5
Branch + PR + protection	Branch, PR #1, #2, screenshot	Gambar 5.5.2
Secrets tidak di repo	GitHub Secrets, `.gitignore`	Gambar 5.5.3

Bab 6

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Buku Servis berhasil dibangun sebagai aplikasi web terintegrasi. API Laravel 12 dengan Sanctum melindungi seluruh endpoint selain login; 16 tes fitur lulus dengan 38 asersi. Frontend Vue 3 memakai Axios dan mengambil alamat API dari `VITE_API_URL`, sehingga kode yang sama dapat berjalan di lokal dan di Docker. Docker Compose menjalankan MySQL, backend FrankenPHP, dan frontend Nginx pada satu network. Data bertahan setelah container dihentikan karena memakai volume bernama.

FrankenPHP terbukti dipakai: log startup mencatat PHP 8.4.27 dan access log mencatat permintaan API, sedangkan header respons memuat `Server: FrankenPHP Caddy`. Pipeline GitHub Actions menjalankan tes dan build pada branch fitur, `dev`, serta pull request; deploy hanya berjalan pada push ke `main` dan menghasilkan image GHCR bertag commit SHA. Branch protection pada `main` dan `dev` mewajibkan pull request serta status check, dan rahasia disimpan di GitHub Secrets.

Pekerjaan ini juga menunjukkan bahwa integrasi tidak selesai setelah kode CRUD berjalan. Dua masalah nyata, yaitu DNS Nginx dan redirect tamu ke route login, baru muncul saat sistem dijalankan sebagai container dan diuji dari proxy. Keduanya diselesaikan dengan konfigurasi kecil dan tes regresi.

6.2 Saran

Untuk pengembangan berikutnya:

1. Tambahkan pengujian UI otomatis dengan Playwright agar alur login, tambah, ubah, dan hapus tidak hanya diuji manual.
2. Jalankan Postman Collection dengan Newman di pipeline agar kontrak API diperiksa pada setiap pull request.
3. Tambahkan fitur pengingat servis berdasarkan interval kilometer atau tanggal, serta ekspor catatan ke PDF/CSV.
4. Pisahkan database production dari database demo dan gunakan migrasi terpisah untuk data contoh.

5. Gunakan secret manager atau environment terenkripsi untuk deployment di server nyata, bukan hanya GitHub Secrets.
6. Tambahkan rate limiting pada login dan audit log untuk permintaan yang mengubah data.

DAFTAR PUSTAKA

Bibliografi

[1] Laravel. *Laravel 12.x Documentation*. https://laravel.com/docs/12.x, diakses 24 Oktober 2026.

[2] Laravel. *Laravel Sanctum*. https://laravel.com/docs/12.x/sanctum, diakses 24 Oktober 2026.

[3] Vue.js. *Vue 3 Documentation*. https://vuejs.org/guide/, diakses 24 Oktober 2026.

[4] Axios. *Axios Documentation*. https://axios-http.com/docs/intro, diakses 24 Oktober 2026.

[5] Docker. *Docker Compose Documentation*. https://docs.docker.com/compose/, diakses 24 Oktober 2026.

[6] FrankenPHP. *FrankenPHP Documentation*. https://frankenphp.dev/docs/, diakses 24 Oktober 2026.

[7] GitHub. *GitHub Actions Documentation*. https://docs.github.com/en/actions, diakses 24 Oktober 2026.

[8] GitHub. *Working with the Container Registry*. https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry, diakses 24 Oktober 2026.

[9] GitHub. *About protected branches*. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches, diakses 24 Oktober 2026.

[10] Postman. *Postman Collection Format*. https://schema.getpostman.com/json/collection/v2.1.0/docs/index.html, diakses 24 Oktober 2026.

LAMPIRAN

A.1 Keluaran Tes

```
 PASS  Tests\Unit\ExampleTest
OK that true is true                                    0.01s

 PASS  Tests\Feature\AuthTest
OK register endpoint is not exposed                     1.24s
OK login with invalid email returns 422                 0.05s
OK login with valid credentials returns token           0.25s
OK login with wrong password returns 401                0.01s
OK logout revokes the token                             0.03s
OK me without token returns 401                         0.02s

 PASS  Tests\Feature\ExampleTest
OK the application returns a successful response        0.05s

 PASS  Tests\Feature\ServiceLogTest
OK guest gets 401 on every endpoint                     0.02s
OK guest without json accept header still gets 401      0.01s
OK index lists only own logs                            0.02s
OK store creates log and returns 201                    0.02s
OK store with invalid data returns 422                  0.02s
OK show returns own log but 404 for other users log     0.01s
OK update changes the log                               0.01s
OK destroy deletes the log                              0.01s

```

```
  Tests:    16 passed (38 assertions)
  Duration: 2.15s
```

Kode A.1: Keluaran lengkap `php artisan test`

A.2 Keluaran `docker compose ps`

```
NAME                IMAGE                  COMMAND                  SERVICE
    CREATED             STATUS                       PORTS
uts-pl-backend-1    uts-pl-backend:local    "entrypoint frankenp..."   backend
    About an hour ago   Up About an hour (healthy)   0.0.0.0:8000->8000/tcp, [::]:8000-
    >8000/tcp
uts-pl-db-1         mysql:8.4.13            "docker-entrypoint.s..."   db
    About an hour ago   Up About an hour (healthy)   3306/tcp, 33060/tcp
uts-pl-frontend-1   uts-pl-frontend:local   "/docker-entrypoint...."   frontend
    About an hour ago   Up About an hour (healthy)   0.0.0.0:8080->80/tcp, [::]:8080-
    >80/tcp
```

Kode A.2: Keluaran `docker compose ps` setelah semua container sehat

A.3 Bukti Log FrankenPHP

```
STARTUP: {"logger":"frankenphp","msg":"FrankenPHP started","php_version":"8.4.27","
    total_threads":48,"worker_threads":0,"max_threads":48,"max_requests":0}
GET /up -> 200
POST /api/login -> 200
POST /api/service-logs -> 422
GET /up -> 200
GET /up -> 200
GET /up -> 200
POST /api/login -> 200
POST /api/service-logs -> 201
GET /api/service-logs -> 200
GET /api/service-logs/7 -> 200
PUT /api/service-logs/7 -> 200
POST /api/service-logs -> 422
GET /api/service-logs -> 401
DELETE /api/service-logs/7 -> 200
POST /api/logout -> 200
GET /up -> 200
GET /up -> 200
GET /up -> 200
GET /up -> 200
```

Kode A.3: Startup dan access log FrankenPHP (tanpa warna)

A.4 Bukti Persistensi

```
SEBELUM DOWN: id uji = 8, jumlah catatan = 6
docker compose down
docker compose up -d
backend /up -> 200
docker compose ps:
NAME                IMAGE                   COMMAND                  SERVICE
    CREATED         STATUS                          PORTS
uts-pl-backend-1    uts-pl-backend:local    "entrypoint frankenp..."   backend    9
    seconds ago   Up 3 seconds (health: starting)   0.0.0.0:8000->8000/tcp, [::]:8000-
    >8000/tcp
uts-pl-db-1         mysql:8.4.13            "docker-entrypoint.s..."   db         9
    seconds ago   Up 8 seconds (healthy)           3306/tcp, 33060/tcp
uts-pl-frontend-1   uts-pl-frontend:local   "/docker-entrypoint...."   frontend   9
    seconds ago   Up 2 seconds (health: starting)   0.0.0.0:8080->80/tcp, [::]:8080-
    >80/tcp
SESUDAH UP: jumlah catatan = 6; data id=8 ditemukan = True; service_type = Uji
    persistensi volume
Pembersihan: DELETE id=8 -> {"message":"Catatan dihapus."}

STATUS AKHIR SETELAH SEHAT:
NAME                IMAGE                   COMMAND                  SERVICE
    CREATED         STATUS                        PORTS
uts-pl-backend-1    uts-pl-backend:local    "entrypoint frankenp..."   backend    2
    minutes ago   Up About a minute (healthy)   0.0.0.0:8000->8000/tcp, [::]:8000-
    >8000/tcp
uts-pl-db-1         mysql:8.4.13            "docker-entrypoint.s..."   db         2
    minutes ago   Up 2 minutes (healthy)       3306/tcp, 33060/tcp
uts-pl-frontend-1   uts-pl-frontend:local   "/docker-entrypoint...."   frontend   2
    minutes ago   Up About a minute (healthy)   0.0.0.0:8080->80/tcp, [::]:8080->80/
    tcp
```

Kode A.4: Transkrip uji persistensi volume

A.5 Transkrip Verifikasi API

```
POST /api/login -> 200
{"user":{"id":1,"name":"Akun Demo","email":"demo@example.com","email_verified_at":null,
    "created_at":"2026-10-24T13:14:17.000000Z","updated_at":"2026-10-24T13
    :14:17.000000Z"},"token":"<token>"}
Token diperoleh: <token>
POST /api/service-logs -> 201
{"data":{"motor":"Vario 125","serviced_at":"2026-10-24","odometer_km":25000,"
    service_type":"Uji verifikasi laporan","cost":60000,"workshop":"AHASS Sleman","
    notes":"data sementara","updated_at":"2026-10-24T13:24:04.000000Z","created_at":"
    2026-10-24T13:24:04.000000Z","id":7}}
```

```
GET /api/service-logs -> 200
{"data":[{"id":7,"motor":"Vario 125","serviced_at":"2026-10-24","odometer_km":25000,"
    service_type":"Uji verifikasi laporan","cost":60000,"workshop":"AHASS Sleman","
    notes":"data sementara","created_at":"2026-10-24T13:24:04.000000Z","updated_at":"
    2026-10-24T13:24:04.000000Z"},{"id":5,"motor":"Vario 125","serviced_at":"2026-10-
    12","odometer_km":24310,"service_type":"Ganti ban belakang","cost":260000,"
    workshop":"Bengkel Pak Min","notes":"Ban lama retak","created_at":"2026-10-24T13
    :14:17.000000Z","updated_at":"2026-10-24T13:14:17.000000Z"},{"id":4,"motor":"Vario
     125","serviced_at":"2026-08-21","odometer_km":23400,"service_type":"Ganti oli
    mesin","cost":55000,"workshop":"AHASS Sleman","notes":null,"created_at":"2026-10-
    24T13:14:17.000000Z","updated_at":"2026-10-24T13:14:17.000000Z"},{"id":3,"motor":"
    Vario 125","serviced_at":"2026-06-09","odometer_km":21800,"service_type":"Servis
    rutin","cost":185000,"workshop":"AHASS Sleman","notes":"Bersihkan CVT, ganti busi",
    "created_at":"2026-10-24T13:14:17.000000Z","updated_at":"2026-10-24T13
    :14:17.000000Z"},{"id":2,"motor":"Vario 125","serviced_at":"2026-04-18","
    odometer_km":19950,"service_type":"Ganti kampas rem depan","cost":70000,"workshop":
    "Bengkel Pak Min","notes":"Kampas sudah tipis","created_at":"2026-10-24T13
    :14:17.000000Z","updated_at":"2026-10-24T13:14:17.000000Z"},{"id":1,"motor":"Vario
     125","serviced_at":"2026-03-02","odometer_km":18420,"service_type":"Ganti oli
    mesin","cost":55000,"workshop":"AHASS Sleman","notes":null,"created_at":"2026-10-
    24T13:14:17.000000Z","updated_at":"2026-10-24T13:14:17.000000Z"}]}
GET /api/service-logs/7 -> 200
{"data":{"id":7,"motor":"Vario 125","serviced_at":"2026-10-24","odometer_km":25000,"
    service_type":"Uji verifikasi laporan","cost":60000,"workshop":"AHASS Sleman","
    notes":"data sementara","created_at":"2026-10-24T13:24:04.000000Z","updated_at":"
    2026-10-24T13:24:04.000000Z"}}
PUT /api/service-logs/7 -> 200
{"data":{"id":7,"motor":"Vario 125","serviced_at":"2026-10-24","odometer_km":25000,"
    service_type":"Uji verifikasi laporan","cost":75000,"workshop":"AHASS Sleman","
    notes":"data sementara","created_at":"2026-10-24T13:24:04.000000Z","updated_at":"
    2026-10-24T13:24:04.000000Z"}}
POST /api/service-logs (tidak valid) -> 422
{"message":"Motor wajib diisi. (and 3 more errors)","errors":{"motor":["Motor wajib
    diisi."],"odometer_km":["Kilometer harus berupa angka bulat."],"service_type":["
    Jenis servis wajib diisi."],"cost":["Biaya tidak boleh kurang dari 0."]}}
GET /api/service-logs tanpa token -> 401
{"message":"Unauthenticated."}
DELETE /api/service-logs/7 -> 200
{"message":"Catatan dihapus."}
POST /api/logout -> 200
{"message":"Berhasil keluar."}
```

Kode A.5: Transkrip curl; token disamarkan

A.6 Tautan Penting

- Repositori: https://github.com/Ryzen0000001/uts-pl-24-535196-SV-24171

- Pull request #1: https://github.com/Ryzen0000001/uts-pl-24-535196-SV-24171/pull/1
- Pull request #2: https://github.com/Ryzen0000001/uts-pl-24-535196-SV-24171/pull/2
- Package backend: https://github.com/users/Ryzen0000001/packages/container/package/uts-pl-24-535196-sv-24171-backend
- Package frontend: https://github.com/users/Ryzen0000001/packages/container/package/uts-pl-24-535196-sv-24171-frontend
- Postman Collection: https://github.com/Ryzen0000001/uts-pl-24-535196-SV-24171/blob/main/postman/buku-servis.postman_collection.json